Universidad Surcolombiana
Facultad de Ingeniería · Programa de Ingeniería de Software

DOCUMENTO DE ARQUITECTURA DE SOFTWARE (SAD)

Plomada

Programación semanal, jornada y evidencia fotográfica para el mantenimiento locativo de conjuntos cerrados

Curso	Ingeniería de Software Orientada a Objetos
Docente	Eduardo Martínez Vidal
Estudiantes	Brayan Toro Bustos Juan Camilo Mora Castañeda Pablo Trujillo Artunduaga
Lugar y fecha	Neiva, noviembre de 2026
Versión	1.0

Contenido

Cómo leer este documento 3

Contexto 4

- Resumen ejecutivo 4
- Título y descripción del proyecto 4
- Metas del negocio 4
- Stakeholders 4
- Propósito y escenarios funcionales 5

1. Atributos de calidad priorizados 6

- Supuestos de volumen 6
- Lo que el taller pide medir 6
- Propiedades sistémicas 6
- Escenarios de calidad priorizados 7
- Árbol de utilidades 9

2. Limitaciones técnicas y de negocio 11

- Restricciones 11
- Deficiencias clave 11

3. Casos de uso 13

4. Escenarios críticos 15

- E1. Sincronización de evidencias capturadas sin conexión 15
- E2. Registro de la jornada con horas capturadas sin conexión 18
- E3. Revisión del supervisor 18
- E4. Publicación de la programación y envío del correo 18

5. Estilo arquitectónico 20

- Decisión 20
- Vista general 21
- Estructura de un módulo 21
- Dependencias entre módulos 22
- Principios aplicados 23
- Alternativas descartadas 25
- Cliente 25

6. Diagrama de componentes 26

7. Diagrama entidad relación 28

8. Justificación de la base de datos 31

9. Diagrama de despliegue 32

- Inventario de recursos 34

10. Tecnologías 35

11. Decisiones críticas 36

001. Monolito modular con arquitectura hexagonal 36
002. PWA que funciona primero sin conexión 36
003. PERN con TypeScript, workspace de pnpm y paquete shared 37
004. Contrato REST expresado como esquemas zod compartidos 38
005. Escrituras idempotentes con UUID del cliente, dos horas por evento y estado derivado 38
006. Token de acceso corto y refresh rotativo 39
007. Subida multipart, disco local detrás de un puerto y compresión en el cliente 40
008. Alcance mínimo vertical y prioridades P0 a P2 40
009. Seguimiento por polling 41
010. Grafo de dependencias entre módulos sin ciclos 41
011. Almacenamiento local en el cliente PWA con Dexie.js 42
012. Migraciones SQL numeradas con un ejecutor propio 43
013. Panel web de administración como excepción al alcance v1 44
014. Autorización por rol en el router, desvío de AGENTS.md 45
015. El contrato no fija el dominio del correo de los operarios 46
016. El día de negocio se cuenta en hora de Bogotá 47

Anexo A. Método de priorización 48

Anexo B. Glosario 49

Personas y lugares 49
Trabajo programado 49
Evidencia y revisión 49
Jornada 50
Sincronización 50
Arquitectura 50

Anexo C. Verificación y defectos conocidos 51

Pruebas y CI 51
Guion de aceptación 51
Cifras de la prueba de ejecución 52
Estado de los defectos 52
Defectos conocidos sin corregir 53

Cómo leer este documento

El taller pide un documento de arquitectura con once ítems. Este documento los presenta con su misma numeración, del 1 al 11, precedidos de un capítulo de contexto que sigue el formato del documento de arquitectura (SAD) del curso: resumen ejecutivo, metas del negocio, stakeholders y escenarios funcionales. Las demás secciones propias del SAD quedan dentro del ítem al que corresponden.

Pide el taller	Sección	Partes del formato SAD incluidas
Contexto	Contexto	Resumen ejecutivo, descripción, metas del negocio, stakeholders, propósito y escenarios funcionales
1. Atributos de calidad priorizados	1	Propiedades sistémicas (ISO/IEC 9126), escenarios de calidad priorizados, árbol de utilidades
2. Limitaciones técnicas o de negocio	2	Restricciones, deficiencias clave
3. Diagramas de caso de uso	3	
4. Escenarios críticos	4	Formato de escenarios operativos
5. Estilo arquitectónico	5	Estilo arquitectónico propuesto, principios aplicados
6. Diagrama de componentes o paquetes	6	
7. Diagrama entidad relación	7	
8. Justificación de la base de datos	8	
9. Diagrama de despliegue	9	Diagrama de despliegue, inventario de recursos
10. Tecnologías utilizadas	10	
11. Decisiones críticas	11	Una nota por decisión: problema, opciones evaluadas, opción seleccionada y por qué
Método de priorización	Anexo A	QAW
Vocabulario	Anexo B	Glosario
Verificación	Anexo C	Pruebas, cifras medidas, estado de los defectos y defectos conocidos

Contexto

Resumen ejecutivo

Una empresa de mantenimiento locativo presta servicios de piscina, pintura, jardinería y otros trabajos a conjuntos cerrados de propiedad horizontal. Cada semana, el administrador programa a unos 150 operarios indicando, de lunes a viernes, el conjunto donde trabajan y las actividades que deben realizar. Hoy no hay un registro confiable de cuándo llegó cada operario, cuándo empezó y terminó cada actividad, ni de cómo quedó el trabajo, y el supervisor que lo revisa en persona no tiene dónde dejar su decisión.

Plomada es el sistema que cierra ese ciclo: publica la programación semanal y la envía por correo a cada operario, registra la jornada, recoge las fotos de antes, durante y después de cada actividad y deja constancia de la revisión del supervisor. La condición que domina el diseño es que los operarios trabajan con su celular personal, de gama baja o media, sin datos móviles y en lugares sin internet garantizado. Por eso el sistema funciona primero sin conexión: todo lo que hace el operario queda guardado en su teléfono y se sincroniza cuando hay red, sin perder ni duplicar nada.

La arquitectura elegida es un monolito modular con arquitectura hexagonal dentro de cada módulo, con un cliente web progresivo (PWA) que funciona sin conexión, una base de datos PostgreSQL y almacenamiento de fotos en disco. Se despliega en local, como permite el taller.

Título y descripción del proyecto

Título: Plomada, sistema de programación, jornada y evidencia fotográfica para el mantenimiento locativo de conjuntos cerrados.

Descripción: el sistema atiende cuatro roles. El administrador gestiona usuarios y, junto con los administrativos, crea y publica la programación semanal, mantiene los centros de trabajo y las hojas de vida, y consulta el seguimiento. El supervisor revisa en sitio las actividades con su evidencia y las aprueba o las rechaza con una observación. El operario consulta su programación, registra la entrada y la salida del centro y el inicio y el fin de cada actividad, y toma las fotos, todo desde su celular y sin depender de la conexión. El nombre viene de la plomada, la herramienta con la que se verifica que un muro quedó a nivel: el supervisor contrasta el trabajo contra la evidencia.

Metas del negocio

1. Contar con evidencia fechada y verificable del trabajo realizado en cada actividad, con fotos de antes, durante y después.
2. Controlar el horario de cada operario: entrada y salida del centro de trabajo, e inicio y fin de cada actividad.
3. Programar la semana y comunicarla a cada operario por su correo institucional, sin llamadas ni mensajes sueltos.
4. Conocer el avance del día por centro de trabajo y por operario.
5. Registrar la revisión del supervisor con su decisión y su observación, como respaldo ante el conjunto cerrado.
6. Centralizar las hojas de vida de los operarios.

Stakeholders

Stakeholder	Justificación
Gerencia de la empresa (administrador)	Responde ante los conjuntos por la calidad del servicio. Espera control del horario, evidencia del trabajo y una programación que llegue a todos sin esfuerzo.

Stakeholder	Justificación
Administrativos (unos 20)	Programan la semana, mantienen centros y hojas de vida y consultan el avance. Esperan rapidez y pocas operaciones.
Supervisores (unos 30)	Revisan el trabajo en persona. Esperan ver la evidencia en su tablet o teléfono y registrar su decisión en sitio.
Operarios (unos 150)	Ejecutan las actividades con su propio celular. Esperan una aplicación sencilla que no gaste datos, no pierda su trabajo y no exija un teléfono nuevo.
Conjuntos cerrados (clientes)	No usan el sistema en esta versión, pero son la razón de la evidencia: el respaldo ante un reclamo sale de aquí.
Docente del curso	Evalúa que la arquitectura se sustente en los principios vistos y que al menos dos módulos funcionen en local.
Equipo de desarrollo	Tres estudiantes: arquitectura y documentación (Brayan Toro), backend y datos (Juan Camilo Mora) y cliente móvil y sincronización (Pablo Trujillo).

Propósito y escenarios funcionales

El propósito es que la empresa sepa, con evidencia, qué se hizo, dónde y cuándo, aunque quien lo hizo haya trabajado sin conexión. Del enunciado se desprenden estos escenarios funcionales, que guían los escenarios de calidad del ítem 1:

1. Un operario sin conexión puede consultar su programación, registrar horas y tomar fotos durante al menos 48 horas.
2. Lo capturado sin conexión llega al servidor una sola vez, aunque el envío se interrumpa y se repita.
3. Cada hecho conserva la hora en que ocurrió y la hora en que llegó al servidor.
4. El supervisor revisa la evidencia desde una tablet o un teléfono y decide en sitio; un rechazo exige observación.
5. Al publicar la programación, cada operario recibe un correo con sus asignaciones sin que el envío retrase la publicación.
6. El sistema soporta 200 cuentas y el pico del cierre de jornada, con 150 operarios sincronizando en media hora.
7. Solo entran usuarios autenticados con su correo institucional, y cada rol ve y hace únicamente lo que le corresponde.
8. Las fotos no se reemplazan ni se borran desde la aplicación.
9. El seguimiento del día refleja lo que llegó al servidor con un retraso máximo de 15 segundos.
10. Cambiar el almacenamiento de fotos o el servidor de correo no exige modificar las reglas de negocio.

1. Atributos de calidad priorizados

Supuestos de volumen

El enunciado no da cifras de uso, así que los objetivos se calculan sobre supuestos explícitos del equipo. Se asumen unas 200 cuentas (1 administrador, 20 administrativos, 30 supervisores y 150 operarios), hasta 6 actividades por operario al día y 3 fotos por actividad. Eso son 150 × 6 × 3 = 2.700 fotos diarias; con unos 400 KB por foto tras la compresión, alrededor de 1,1 GB al día, 5,4 GB a la semana y unos 280 GB al año. Si el 80 % de los operarios sincroniza en la media hora posterior al cierre de la jornada, llegan en promedio 1,2 fotos por segundo, y el sistema se diseña para sostener 5 por segundo, cuatro veces ese promedio. Los 30 supervisores consultando cada 12 segundos suman unas 2,5 peticiones por segundo.

Lo que el taller pide medir

Aspecto	Objetivo	Cómo lo logra la arquitectura	Escenario
Tiempos de respuesta	En el servidor, p95 de 500 ms en consultas y registros, 1 s en el inicio de sesión (el hash es lento a propósito) y 2 s para procesar una foto de 400 KB. En el celular, toda acción del operario responde en menos de 100 ms.	Las acciones del operario terminan contra el almacenamiento local; la sincronización corre aparte.	EC-05
Número de usuarios	200 cuentas. En el pico, 150 operarios sincronizando en 30 minutos (5 fotos por segundo sostenidas) y 30 supervisores consultando a la vez. Crecer al doble sin cambiar la arquitectura.	Un solo proceso que escala verticalmente, con una carga muy por debajo de sus límites.	EC-05
Autenticación y cifrado	Correo institucional y contraseña con hash argon2id. Token de acceso firmado de 15 minutos y token de renovación rotativo de 7 días, guardado solo como hash. TLS 1.2 o superior en tránsito en producción, que queda fuera de v1: la demostración local corre por HTTP en `localhost`. Fotos servidas solo con token. Cifrado en reposo delegado al disco del sistema operativo.	Módulo `identidad`; el rol se comprueba en el router y la propiedad del recurso en los casos de uso (notas 006 y 014).	EC-03, EC-04
Tiempo en línea	99,5 % en horario laboral (lunes a viernes, 6:00 a 18:00), unas 1,3 horas de caída al mes. Tras una caída, el servicio vuelve en menos de 5 minutos.	El cliente trabaja sin servidor, así que la disponibilidad que percibe el operario es mayor que la del servidor.	EC-01, EC-08

La disponibilidad del servidor no está entre las primeras prioridades, y es a propósito. Como el operario trabaja y acumula en su celular aunque el servidor esté caído, una caída corta retrasa la sincronización pero no detiene a nadie. Eso permite un despliegue local de un solo servidor sin redundancia, que es lo que admite el taller.

Propiedades sistémicas

El curso usa el modelo de calidad ISO/IEC 9126, y este documento conserva su vocabulario. Su sucesora, ISO/IEC 25010, agrupa las mismas ideas con otros nombres; ninguna conclusión cambia por ello.

Característica	Subcaracterística	Qué significa en Plomada	Prioridad	Escenario
Fiabilidad	Tolerancia a fallos	Trabajar sin conexión y sin servidor sin perder ni duplicar datos	Alta	EC-01
Fiabilidad	Capacidad de recuperación	Tras una caída, el servidor vuelve y las colas de los celulares se vacían solas	Media	EC-08
Funcionalidad	Precisión	Cada hecho con su hora capturada y su hora recibida; el estado derivado de los hechos	Alta	EC-02
Funcionalidad	Seguridad	Autenticación, autorización por rol y credenciales protegidas	Alta	EC-03, EC-04
Funcionalidad	Interoperabilidad	Correo por SMTP y funcionamiento en navegadores de Android e iOS	Media	EC-10
Eficiencia	Comportamiento en el tiempo	Respuestas rápidas aun en el pico de cierre de jornada	Media	EC-05

Característica	Subcaracterística	Qué significa en Plomada	Prioridad	Escenario
Eficiencia	Utilización de recursos	Poca memoria, poco almacenamiento y pocos datos en celulares de gama baja	Alta	EC-06
Mantenibilidad	Cambiabilidad	Cambiar la infraestructura sin tocar las reglas de negocio	Media	EC-07
Mantenibilidad	Facilidad de prueba	Dominio y casos de uso probados sin base de datos	Media	EC-07
Usabilidad	Operatividad y facilidad de aprendizaje	Pocas acciones por actividad, para usuarios con poca experiencia digital	Media	EC-09
Portabilidad	Adaptabilidad y facilidad de instalación	Cualquier navegador moderno, en teléfono o tablet, sin tienda de aplicaciones	Baja	EC-10

Escenarios de calidad priorizados

Cada escenario sigue la estructura de seis partes del curso: fuente, estímulo, artefacto, ambiente, respuesta y medida de la respuesta. A diferencia de una descripción general, la medida es una cifra que se puede comprobar.

EC-01. Tolerancia a la desconexión. Prioridad alta. Stakeholder: operarios y gerencia.

Fuente	Operario en un centro de trabajo sin señal
Estímulo	Registra su jornada y toma las fotos de sus actividades durante dos días sin conexión; al volver la red, el enlace se corta varias veces
Artefacto	PWA, cola local y API de sincronización
Ambiente	Sin conexión hasta 48 horas y luego con red intermitente
Respuesta	Todo queda guardado en el celular; al volver la red, la cola se envía en orden de captura y cada envío interrumpido se repite sin duplicar
Medida de la respuesta	Cero fotos perdidas y cero duplicadas. Con una conexión estable de 1 Mbps, la cola de una jornada (18 fotos) se vacía en menos de 3 minutos

EC-02. Precisión de los registros. Prioridad alta. Stakeholder: gerencia y supervisores.

Fuente	Celular con el reloj desajustado y conexión tardía
Estímulo	El registro de fin de una actividad llega al servidor cuatro horas después y antes que el de inicio
Artefacto	Módulos `jornada` y `programacion`
Ambiente	Sincronización diferida, fuera de orden
Respuesta	El servidor guarda la hora capturada y la recibida sin corregir ninguna, y deriva el estado de la actividad a partir de los hechos, no del orden de llegada
Medida de la respuesta	El 100 % de los hechos tiene ambas horas. El estado final es el mismo con cualquier orden de llegada, comprobado por pruebas automatizadas

EC-03. Control de acceso por rol. Prioridad alta. Stakeholder: gerencia.

Fuente	Usuario sin sesión, o un operario autenticado
Estímulo	Intenta revisar una actividad o crear un usuario
Artefacto	API, casos de uso de `evidencias` e `identidad`
Ambiente	Operación normal
Respuesta	La petición se rechaza con `no_autenticado` o `sin_permiso` y no cambia ningún dato
Medida de la respuesta	El 100 % de las rutas, salvo el inicio de sesión y la de salud, exige token; cada regla de rol tiene una prueba automatizada: `backend/src/matriz-roles.test.ts` recorre el registro `rutas` de `shared` y comprueba el 401 sin token y el 403 `sin_permiso` de cada rol no permitido

EC-04. Protección de credenciales. Prioridad alta. Stakeholder: gerencia y operarios.

Fuente	Persona con acceso a una copia de la base de datos
Estímulo	Lee las tablas de usuarios y de sesiones
Artefacto	Base de datos, módulo `identidad`
Ambiente	Operación normal, respaldo filtrado
Respuesta	Encuentra hashes argon2id de las contraseñas y hashes de los tokens de renovación, nunca valores utilizables
Medida de la respuesta	Ninguna credencial ni token vigente recuperable desde la base. En producción todo el tráfico iría por TLS 1.2 o superior; queda fuera de v1 y la demostración local corre por HTTP en `localhost`

EC-05. Respuesta en el pico de cierre de jornada. Prioridad media. Stakeholder: supervisores y administrativos.

Fuente	30 supervisores y 150 operarios
Estímulo	Los supervisores consultan la cola de revisión cada 12 segundos mientras los operarios sincronizan sus fotos
Artefacto	API y base de datos
Ambiente	Pico de carga: los 30 minutos posteriores al cierre de la jornada
Respuesta	Se atienden todas las peticiones sin errores por carga
Medida de la respuesta	p95 de 500 ms en consultas y registros, y de 2 s por foto de 400 KB, sosteniendo 5 fotos por segundo

EC-06. Uso de recursos en gama baja. Prioridad alta. Stakeholder: operarios.

Fuente	Operario con un celular de gama baja
Estímulo	Abre la aplicación por primera vez y luego toma las fotos de una jornada
Artefacto	PWA
Ambiente	Primera visita con conexión; uso posterior sin ella
Respuesta	La aplicación se instala y queda disponible sin conexión; las fotos se comprimen antes de guardarse
Medida de la respuesta	Carga inicial de 500 KB comprimidos como máximo. Fotos de unos 400 KB y 1600 px en su lado mayor, con 1,5 MB como límite que acepta el servidor

EC-07. Cambio de infraestructura. Prioridad media. Stakeholder: equipo de desarrollo y gerencia.

Fuente	Equipo de desarrollo
Estímulo	Se decide guardar las fotos en un servicio de almacenamiento en la nube en vez del disco local
Artefacto	Módulo `evidencias`
Ambiente	Mantenimiento
Respuesta	Se escribe un adaptador nuevo que implementa el puerto `AlmacenamientoEvidencias` y se conecta en la raíz de composición
Medida de la respuesta	Cero cambios en `domain/` y `application/`; las pruebas del dominio siguen pasando sin modificarse

EC-08. Recuperación tras una caída. Prioridad media. Stakeholder: gerencia.

Fuente	Falla del servidor o corte eléctrico
Estímulo	El servidor deja de responder durante una hora en horario laboral
Artefacto	Servidor, API y clientes
Ambiente	Horario laboral
Respuesta	Los operarios siguen trabajando sin conexión; al volver el servidor, las colas se vacían sin intervención
Medida de la respuesta	99,5 % de disponibilidad en horario laboral, servicio restablecido en menos de 5 minutos tras la reparación y ningún dato perdido

EC-09. Operación por un usuario con poca experiencia. Prioridad media. Stakeholder: operarios.

Fuente	Operario con poca experiencia digital
Estímulo	Registra una actividad completa: inicio, tres fotos y fin
Artefacto	PWA
Ambiente	En el centro de trabajo, con o sin conexión
Respuesta	Cada paso es una acción visible desde la actividad del día, con la cámara nativa del teléfono
Medida de la respuesta	La actividad completa toma menos de 2 minutos, con una demostración previa de 5 minutos como única capacitación

EC-10. Instalación sin tienda de aplicaciones. Prioridad baja. Stakeholder: operarios y supervisores.

Fuente	Operario o supervisor con un teléfono Android o iOS
Estímulo	Abre el enlace del sistema por primera vez
Artefacto	PWA
Ambiente	Primera visita con conexión
Respuesta	El navegador ofrece instalar la aplicación en la pantalla de inicio
Medida de la respuesta	Instalada en menos de 1 minuto, sin tienda de aplicaciones ni permisos especiales

Árbol de utilidades

El árbol resume la priorización. Cada hoja lleva dos letras entre paréntesis: la primera es la importancia para el negocio y la segunda la dificultad técnica, cada una alta (A), media (M) o baja (B). Las hojas (A, A) son las que más condicionan la arquitectura.

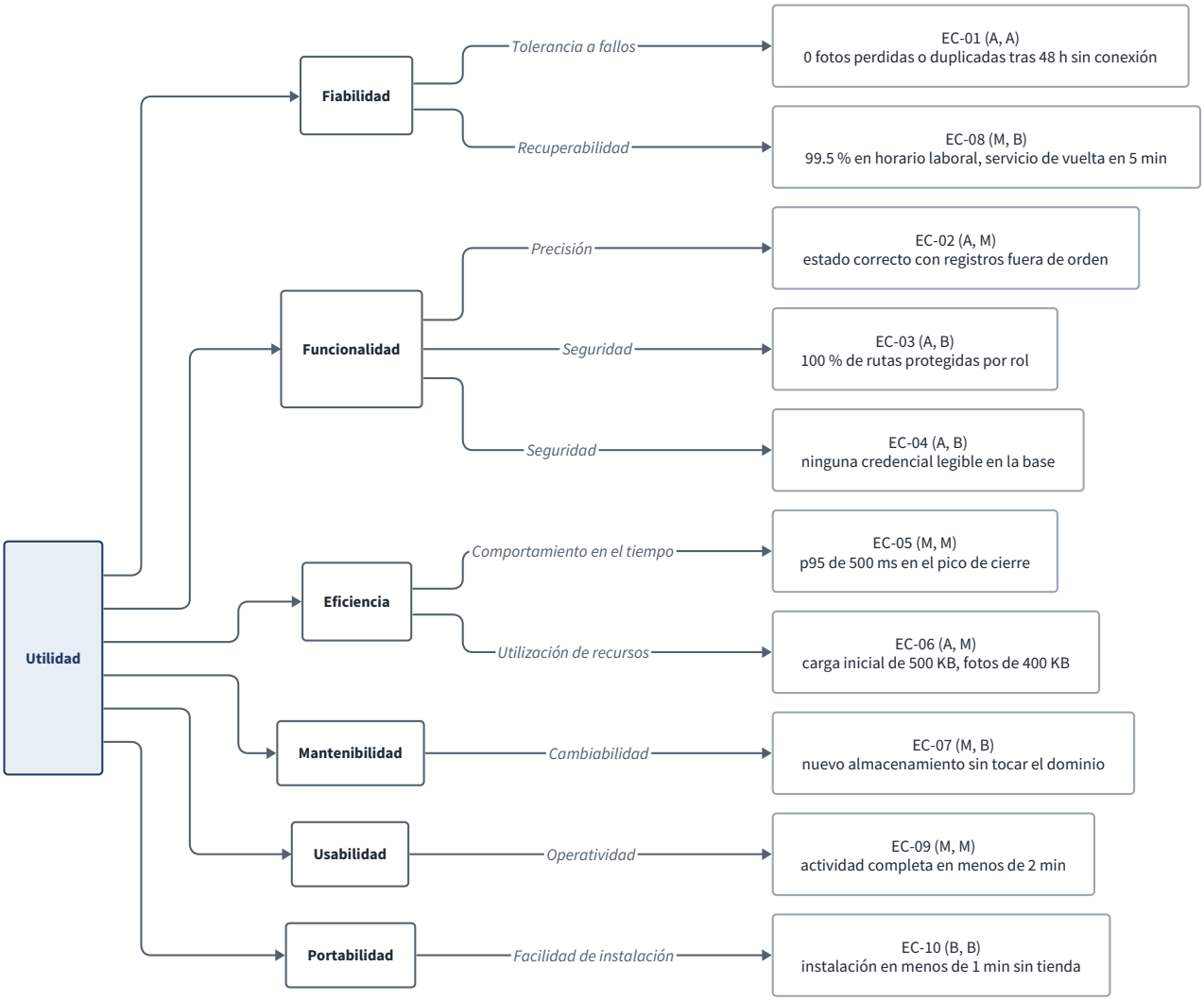


Figura 1. Árbol de utilidades

2. Limitaciones técnicas y de negocio

Restricciones

ID	Restricción	Tipo	Establecida por	Alternativa considerada	Efecto en el diseño
R1	Operarios sin datos móviles ni internet garantizado en el sitio	Negocio	Enunciado	Exigir conexión o pagar datos a los operarios	Cliente que funciona primero sin conexión (nota 002)
R2	Celulares personales de gama baja o media, heterogéneos y sin gestión de la empresa	Tecnología	Enunciado	Aplicación nativa para Android	PWA, compresión de fotos y cámara nativa
R3	Despliegue local	Tecnología	Taller	Servicio en la nube	Un servidor, Docker Compose para la base de datos y el correo
R4	Al menos dos módulos funcionando	Negocio	Taller	Prototipo de todo el sistema	Recorrido mínimo vertical con prioridades P0 a P2 (nota 008)
R5	La IA se permite si cada decisión se sustenta en principios	Negocio	Taller	No usarla	Decisiones registradas y reglas para agentes en `AGENTS.md`
R6	Cada operario necesita un correo institucional, creado por la empresa	Negocio	Enunciado	Usuario y contraseña sin correo	El correo identifica al usuario y recibe la programación
R7	Programación semanal, de lunes a viernes, hecha por la administración	Negocio	Enunciado	Programación diaria	Una programación por semana, sin edición tras publicarla
R8	Tres fotos por actividad: antes, durante y después	Negocio	Enunciado	Una foto final	Tres tipos de evidencia, con más de una foto por tipo
R9	El supervisor revisa en persona	Negocio	Enunciado	Aprobación automática	La revisión registra la decisión del supervisor, no la reemplaza
R10	El service worker y `crypto.randomUUID()` exigen HTTPS o `localhost`	Tecnología	Navegador	Certificado local o túnel HTTPS	La demostración corre en `localhost` con emulación de móvil
R11	Background Sync no existe en iOS	Tecnología	Plataforma	Depender de Background Sync	Sincronización al abrir la aplicación, al volver la red y con un botón
R12	El reloj del celular no es fiable	Tecnología	Dispositivo	Corregir con la hora del servidor	Hora capturada y hora recibida, sin corregir ninguna (nota 005)
R13	Almacenamiento del navegador limitado y que el sistema puede borrar	Tecnología	Navegador	Guardar fotos sin comprimir	Compresión, pedido de almacenamiento persistente y aviso de pendientes
R14	Equipo de tres personas, dos con poca experiencia en Git	Negocio	Equipo	Repositorios separados	Contrato como código, integración continua y carpetas con dueño

Deficiencias clave

Hay riesgos que la arquitectura reduce pero no elimina. Se declaran para que nadie los dé por resueltos.

1. **Conectividad.** El seguimiento solo puede mostrar lo que ya llegó. Si un operario no encuentra red en todo el día, su avance aparece al final de la jornada. El sistema muestra la hora de la última sincronización de cada operario para que el supervisor sepa qué tan actual es lo que ve.
2. **Reloj del celular.** Un reloj muy desajustado produce una hora capturada dudosa que el sistema no detecta. Se muestran las dos horas para que la diferencia sea visible.
3. **Pérdida del celular antes de sincronizar.** Lo que no ha llegado al servidor se pierde con el teléfono o con el borrado de datos del navegador. Se reduce sincronizando en cada oportunidad, pidiendo almacenamiento persistente y mostrando al operario cuántos elementos tiene pendientes.

4. **Un solo servidor.** No hay redundancia. Un daño de disco pierde fotos y datos si no hay respaldo, así que la operación debe incluir una copia diaria de la base de datos y de la carpeta de fotos.
5. **Privacidad de las fotos.** Las fotos pueden mostrar personas de los conjuntos. Solo se sirven a usuarios autenticados, pero la política de cuánto tiempo se conservan la debe definir la empresa.
6. **Dependencia del correo institucional.** Un operario sin correo creado no puede entrar ni recibir su programación; la creación de cuentas es un paso de la empresa, no del sistema.

3. Casos de uso

Los casos de uso se presentan en dos diagramas, uno para la operación en campo y otro para la gestión, porque en uno solo las catorce asociaciones vuelven ilegible la figura. `Usuario` es un actor abstracto: operario, supervisor y administrativo lo especializan, y por eso todos inician sesión. El administrador especializa al administrativo, así que hace todo lo que este hace y además gestiona usuarios.

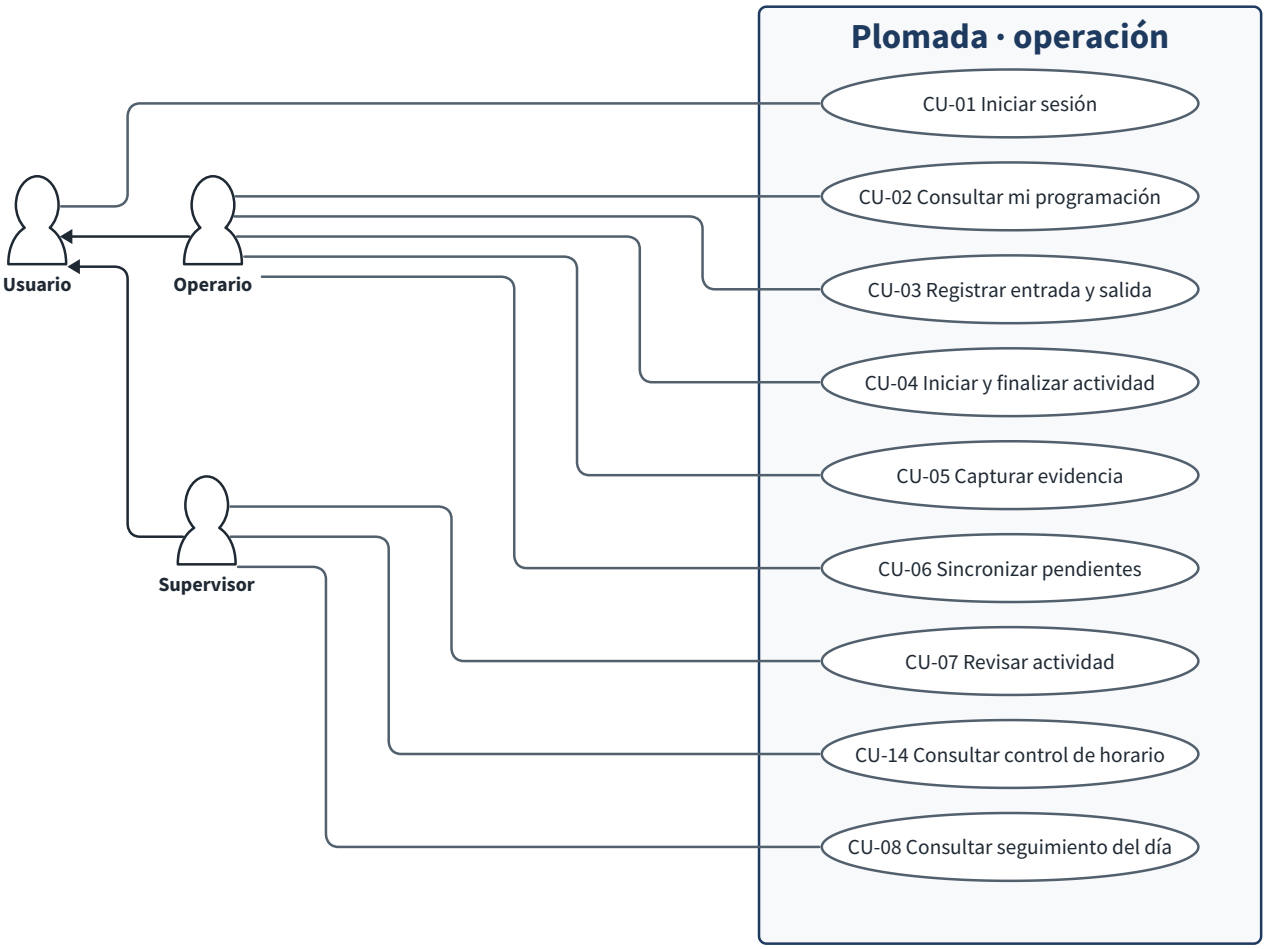


Figura 2. Casos de uso de la operación en campo

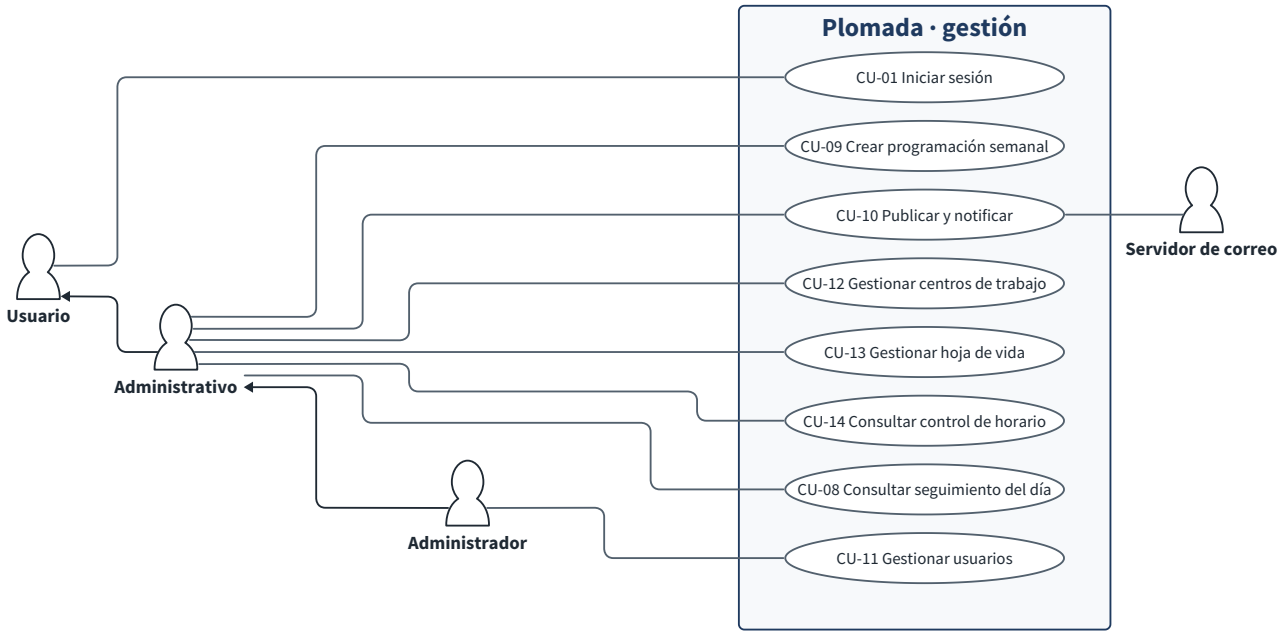


Figura 3. Casos de uso de gestión

Caso de uso	Actor principal	Módulo	Prioridad
CU-01 Iniciar sesión con correo institucional y renovar la sesión	Todos	`identidad`	P0
CU-02 Consultar la programación de la semana, también sin conexión	Operario	`programacion`	P0
CU-03 Registrar la entrada y la salida del centro de trabajo	Operario	`jornada`	P1
CU-04 Registrar el inicio y el fin de una actividad	Operario	`jornada`	P1
CU-05 Tomar las fotos de antes, durante y después de una actividad	Operario	`evidencias`	P0
CU-06 Sincronizar con el servidor lo capturado sin conexión	Operario	`evidencias y jornada`	P0
CU-07 Revisar una actividad y aprobarla o rechazarla con observación	Supervisor	`evidencias`	P0
CU-08 Consultar el seguimiento del día	Supervisor y administrativo	`seguimiento`	P2
CU-09 Crear la programación semanal	Administrativo	`programacion`	P0
CU-10 Publicar la programación y enviar el correo a cada operario	Administrativo	`programacion`	P0
CU-11 Gestionar usuarios	Administrador	`identidad`	P1
CU-12 Gestionar centros de trabajo	Administrativo	`programacion`	P1
CU-13 Gestionar la hoja de vida de un operario	Administrativo	`personal`	P2
CU-14 Consultar el control de horario	Supervisor y administrativo	`jornada`	P1

P0 es lo necesario para la demostración, P1 completa el flujo y P2 se implementa solo si sobra tiempo (nota 008).

4. Escenarios críticos

Se identifican cuatro flujos con mayor impacto en la arquitectura. Cada uno obliga a una decisión que de otro modo sería más barata, y por eso son los que el diseño defiende. Cada ficha sigue el formato de escenarios operativos del curso.

E1. Sincronización de evidencias capturadas sin conexión

Stakeholder	Operarios
Descripción general	El operario toma las fotos de una actividad sin conexión y la aplicación las envía cuando vuelve la red
Lo que hace hoy o quisiera hacer	Hoy no hay registro; quisiera tomar las fotos sin preocuparse por la señal
Disponible al inicio	Sesión iniciada al menos una vez con conexión y programación de la semana descargada
Contexto de la operación	En el centro de trabajo, a menudo en sótanos o zonas sin señal
Cómo debe responder el sistema	Guardar cada foto en el celular al instante y enviarla después, una sola vez, aunque el envío se repita
Salidas	Fotos con sus metadatos en el servidor y estado de la actividad actualizado
Quién usa la salida y para qué	El supervisor, para revisar; la gerencia, como respaldo ante el conjunto

La captura termina contra el almacenamiento local, sin red.

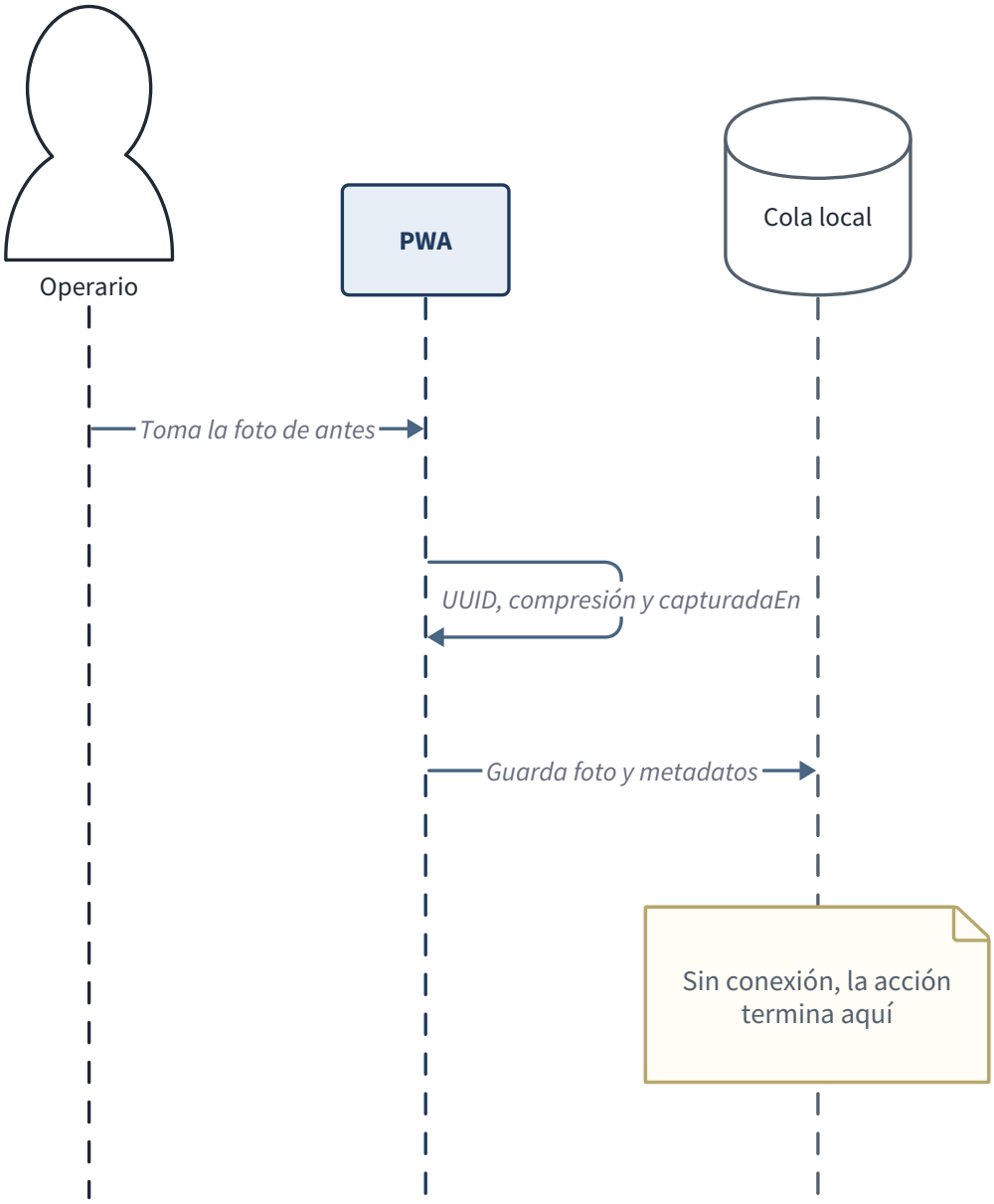


Figura 4. Secuencia de la captura sin conexión

La sincronización primero renueva la sesión y luego vacía la cola elemento por elemento, en orden de captura. Si un corte interrumpe un envío, el cliente no sabe si el servidor lo recibió y simplemente lo repite: la segunda vez el servidor responde 200 con la foto ya guardada y no crea otra.

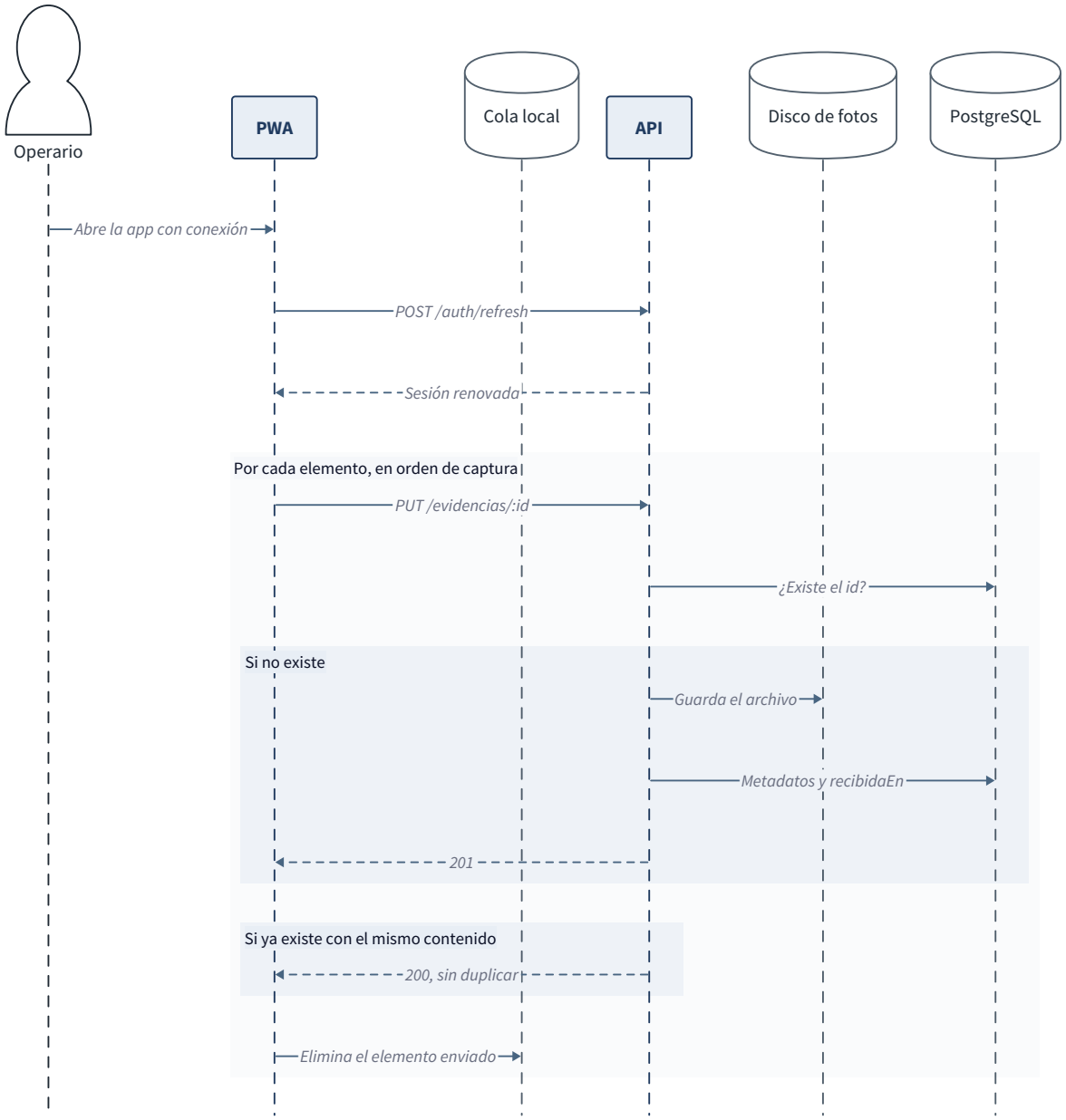


Figura 5. Secuencia del envío de evidencias al volver la conexión

Atributos que lo gobiernan: tolerancia a fallos, precisión y comportamiento en el tiempo. **Impacto en la arquitectura:** obliga a un cliente con almacenamiento y cola propios, a un contrato con escrituras idempotentes y a renovar la sesión antes de subir (notas 002, 005 y 006).

E2. Registro de la jornada con horas capturadas sin conexión

Stakeholder	Gerencia y supervisores
Descripción general	El operario marca la entrada al centro y el inicio y el fin de cada actividad
Lo que hace hoy o quisiera hacer	Hoy el horario se reporta de palabra; la gerencia quisiera un registro creíble
Disponible al inicio	Programación del día en el celular
Contexto de la operación	Los registros pueden llegar horas después y fuera de orden
Cómo debe responder el sistema	Guardar la hora capturada y la recibida sin corregirlas, y derivar el estado de la actividad de los hechos conocidos
Salidas	Control de horario por operario y estado de cada actividad
Quién usa la salida y para qué	Supervisores y administrativos, para controlar el cumplimiento

Atributos: precisión y tolerancia a fallos. **Impacto:** dos horas por evento, estado derivado en vez de almacenado y registros idempotentes por UUID (nota 005). Es lo que permite que el control de horario sea creíble aunque la red no lo sea.

E3. Revisión del supervisor

Stakeholder	Supervisores
Descripción general	El supervisor abre las actividades con evidencia sin revisar, mira las fotos y aprueba o rechaza
Lo que hace hoy o quisiera hacer	Hoy revisa en persona sin dejar constancia; quisiera registrar su decisión en sitio
Disponible al inicio	Al menos una evidencia sincronizada en la actividad
Contexto de la operación	En el conjunto, desde una tablet o un teléfono
Cómo debe responder el sistema	Mostrar las fotos rápido y registrar la decisión; un rechazo exige observación
Salidas	Actividad aprobada o rechazada, con autor, hora y observación
Quién usa la salida y para qué	Gerencia y administrativos, para el seguimiento y como respaldo ante el conjunto

Dos supervisores pueden abrir la misma actividad; la última decisión reemplaza a la anterior. Es aceptable porque la revisión es presencial y poco concurrente. **Atributos:** precisión, seguridad (solo el supervisor decide) y comportamiento en el tiempo. **Impacto:** la foto se sirve por una ruta autenticada con encabezados de caché inmutables, y la cola de revisión es una consulta derivada de los hechos, sin una tabla propia.

E4. Publicación de la programación y envío del correo

Stakeholder	Administrativos y operarios
Descripción general	Al publicar la semana se envía un correo a cada operario con sus asignaciones, unos 150 mensajes
Lo que hace hoy o quisiera hacer	Hoy la programación se comunica por mensajes sueltos; quisieran publicar una vez y que llegue a todos
Disponible al inicio	Programación en borrador con sus asignaciones
Contexto de la operación	Una vez por semana, antes del lunes
Cómo debe responder el sistema	Publicar de inmediato; el correo es un efecto secundario que no bloquea ni hace fallar la publicación
Salidas	Programación publicada y un correo por operario
Quién usa la salida y para qué	Los operarios, para saber dónde y qué trabajar cada día

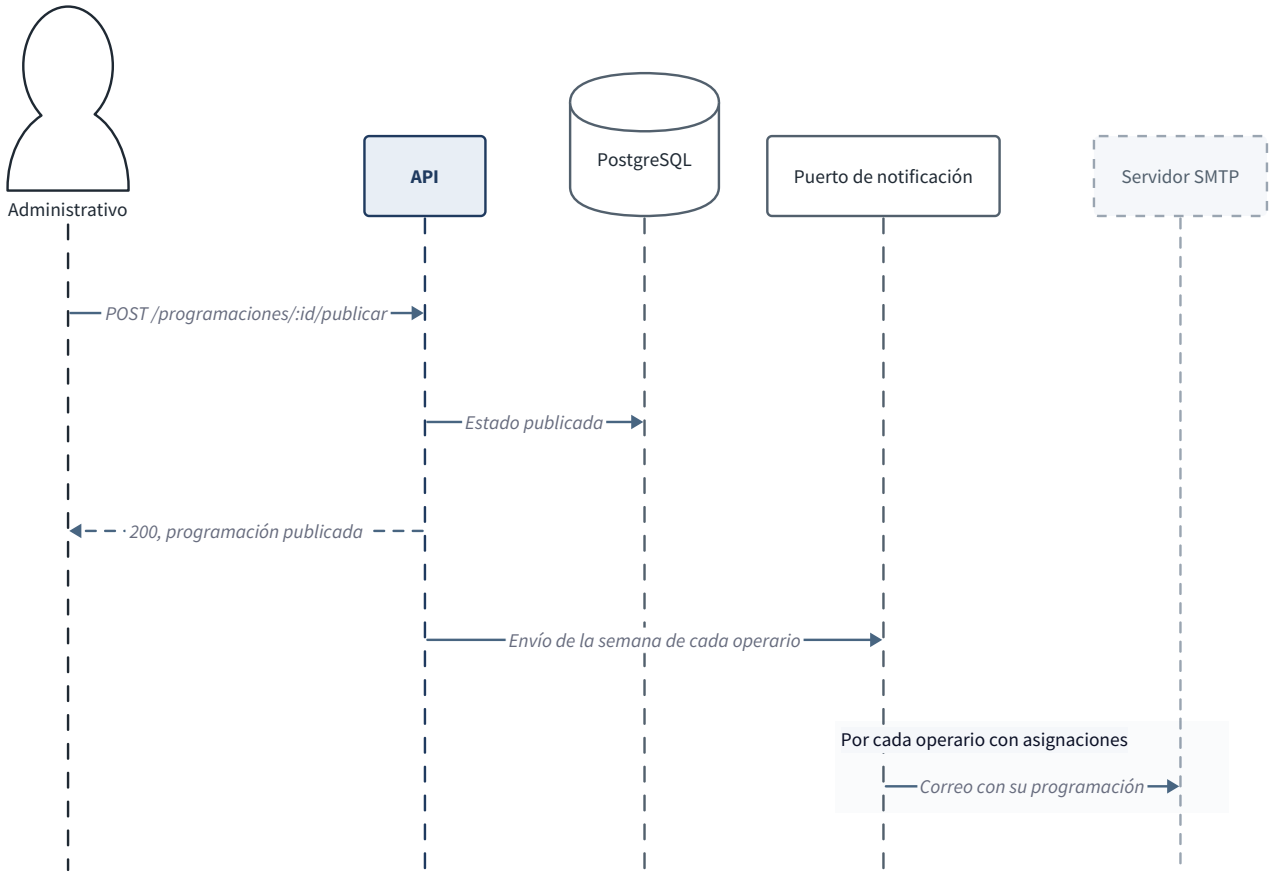


Figura 6. Secuencia de la publicación y el envío de correos

Atributos: cambiabilidad y capacidad de recuperación. **Impacto:** el envío va detrás de un puerto de notificación con un adaptador SMTP, de modo que en desarrollo lo recibe Mailpit y en producción un servidor real, y el dominio no sabe de correo. Un servidor SMTP caído no impide que la programación quede publicada.

5. Estilo arquitectónico

Decisión

El estilo es un monolito modular con arquitectura hexagonal dentro de cada módulo (nota 001). El sistema se despliega como una sola aplicación organizada en seis módulos por capacidad de negocio, y cada módulo separa dominio, casos de uso, puertos y adaptadores. El cliente es una aplicación web progresiva que funciona primero sin conexión.

La razón es la escala real del problema: 200 usuarios, un servidor local y tres desarrolladores. Un monolito evita la red, los despliegues múltiples y la consistencia distribuida que cobrarían los microservicios sin devolver nada a esta escala. La división en módulos con límites estrictos conserva lo que sí importa, que es poder razonar, probar y cambiar cada parte por separado. Y la arquitectura hexagonal aísla las reglas de negocio de las tres cosas que pueden cambiar: la base de datos, el almacenamiento de fotos y el correo.

Vista general

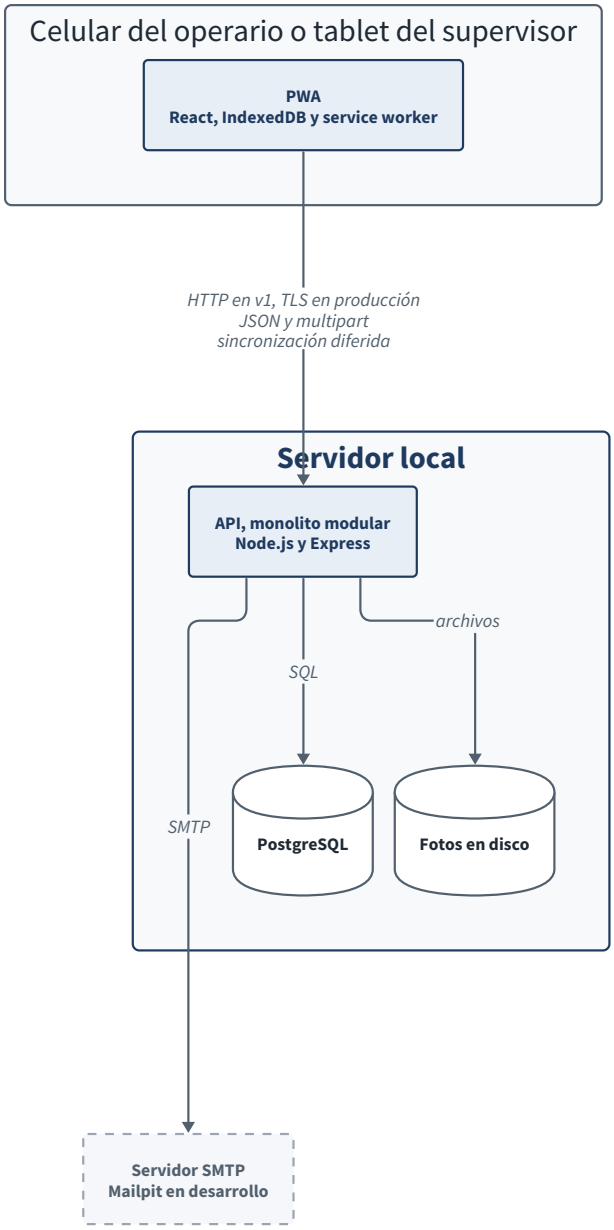


Figura 7. Vista general del sistema

Estructura de un módulo

La dependencia apunta siempre hacia el dominio. Los casos de uso conocen puertos, que son interfaces, y los adaptadores de infraestructura los implementan; el dominio no importa ninguna librería externa.

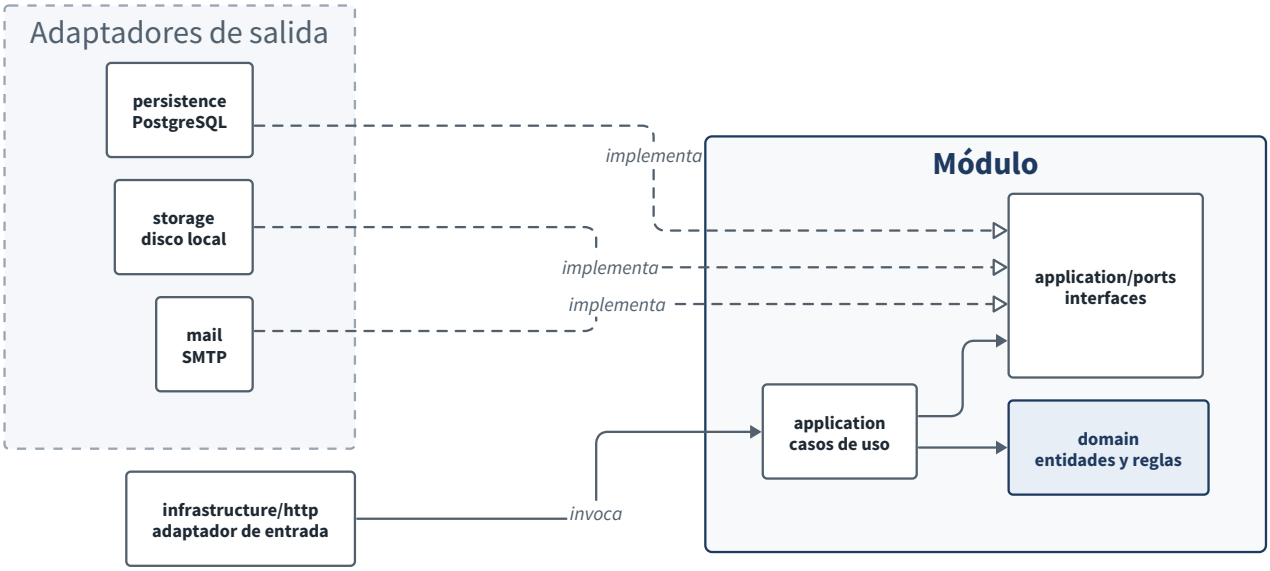


Figura 8. Estructura hexagonal de un módulo

Dependencias entre módulos

Los módulos forman un grafo sin ciclos. Cada flecha significa "usa la interfaz pública de". Cuando un módulo necesita algo de otro que ya depende de él, declara un puerto y la raíz de composición lo implementa. Es el caso de `HechosDeActividadPort`, que `programacion` declara para conocer el estado de sus actividades sin importar `jornada` ni `evidencias` (nota 010). El diagrama dibuja las aristas que existen hoy en el código: `jornada` y `evidencias` usan la interfaz pública de `programacion`, y `seguimiento` usa la de `jornada` y la de `programacion`. La nota 010 prevé además que `seguimiento` dependa de `evidencias`; el código no lo hace, porque los hechos de las evidencias le llegan a través de `HechosDeActividadPort`. `identidad` y `personal` no importan a ningún módulo y ningún módulo los importa: los datos de usuarios llegan a los demás como funciones que entrega la raíz de composición.

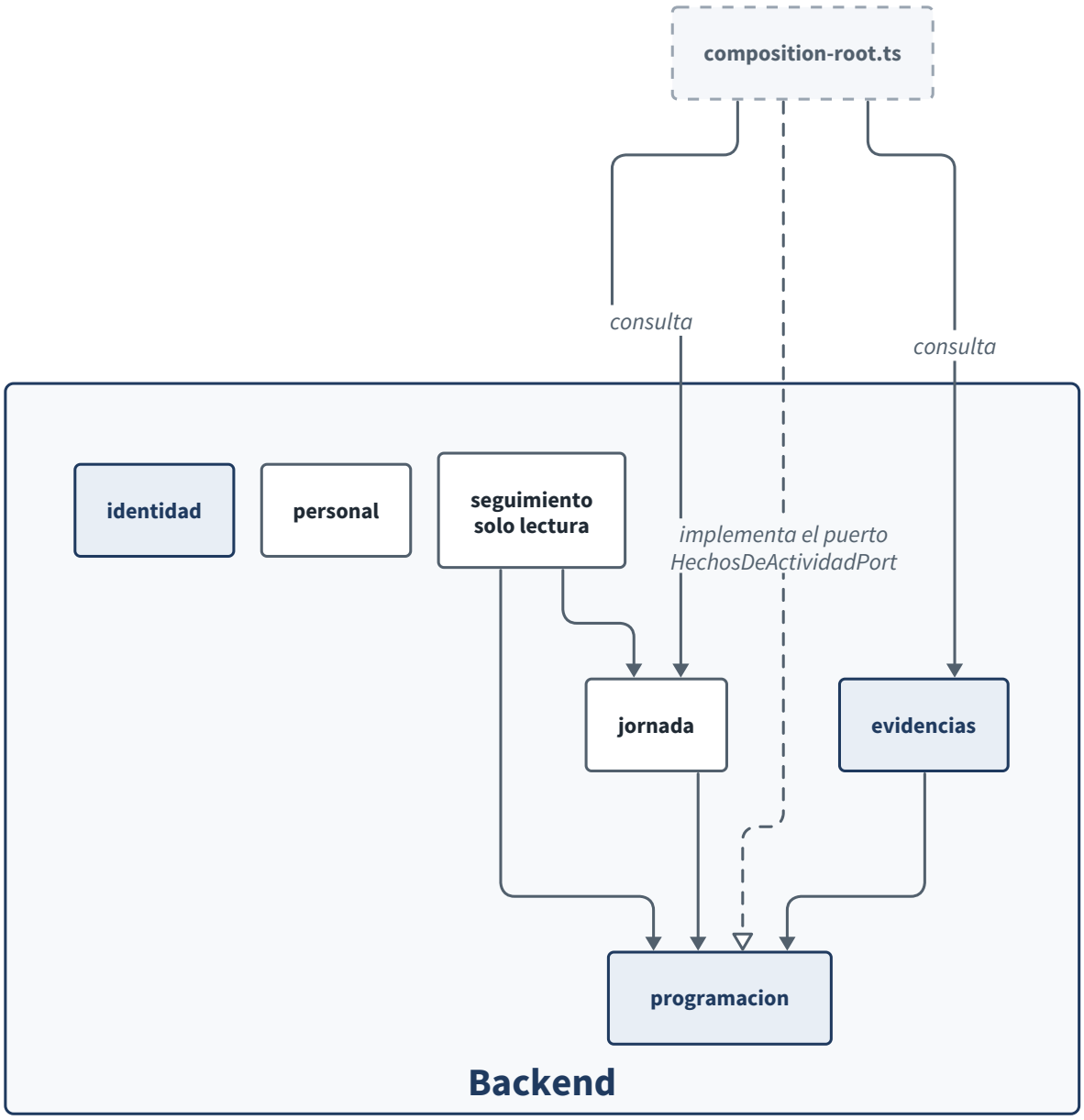


Figura 9. Dependencias entre módulos

Principios aplicados

El curso agrupa los principios de diseño en SOLID y en las prácticas de código limpio (Clean Code). La tabla muestra dónde se aplica cada uno, para que el código se pueda defender contra este documento.

Principio	Cómo se aplica	Dónde se ve
Responsabilidad única (S)	Cada módulo agrupa una capacidad de negocio y cada caso de uso es un archivo con una sola tarea. Los adaptadores solo traducen entre el exterior y el dominio.	`backend/src/modules/evidencias/application/subir-evidencia.use-case.ts:36` y `backend/src/modules/jornada/application/registrar-jornada.use-case.ts:21`

Principio	Cómo se aplica	Dónde se ve
Abierto y cerrado (O)	`SubirEvidenciaUseCase` solo conoce el puerto `AlmacenamientoEvidenciasPort`, así que sirve con cualquiera de sus adaptadores sin modificarlo: hoy existen `DiscoAlmacenamientoEvidencias` y `InMemoryAlmacenamientoEvidencias`. Un almacenamiento en la nube sería un tercero; no existe hoy (nota 007).	`backend/src/modules/evidencias/application/ports/almacenamiento-evidencias.port.ts:1`, `.../infrastructure/storage/disco-almacenamiento-evidencias.ts:5` y `.../in-memory-almacenamiento-evidencias.ts:3`
Sustitución de Liskov (L)	El repositorio en memoria y el de PostgreSQL implementan la misma interfaz, y el compilador verifica que cualquiera sirva donde se pide el puerto. Las dos implementaciones se prueban en suites distintas: la de memoria junto con los casos de uso y la de PostgreSQL contra la base real.	`backend/src/modules/identidad/application/ports/sesion-repository.port.ts:3`, `.../persistence/pg-sesion.repository.ts:5` y `.../in-memory-sesion.repository.ts:4`; pruebas en `identidad.test.ts:20` y `backend/src/adaptadores-pg.test.ts:103`
Segregación de interfaces (I)	Los puertos son pequeños y los declara quien los usa. `programacion` pide solo los hechos que necesita para derivar estados, no todo `jornada` ni todo `evidencias`.	`backend/src/modules/programacion/application/ports/hechos-de-actividad.port.ts:3`; implementado en `backend/src/composition-root.ts:131`; nota 010
Inversión de dependencias (D)	Los casos de uso dependen de interfaces y no de PostgreSQL, del disco ni de SMTP. Las implementaciones se conectan en un único punto.	`backend/src/modules/evidencias/application/subir-evidencia.use-case.ts:37` (el constructor recibe puertos) y `backend/src/composition-root.ts:87`
Nombres con intención	El código usa el vocabulario del negocio definido en el glosario: `Actividad`, `Evidencia`, `RegistroJornada`. El operario no es un tipo propio: es un `Usuario` con rol `operario`.	`shared/src/programacion.ts:29`, `shared/src/evidencias.ts:19`, `shared/src/jornada.ts:46`, `shared/src/usuarios.ts:4`; Anexo B
Funciones pequeñas y sin efectos ocultos	El estado de una actividad lo calcula una función pura a partir de los hechos; registrar un hecho y consultar un estado son operaciones separadas.	`derivarEstado` en `backend/src/modules/programacion/domain/estado-actividad.ts:23`
Manejo explícito de errores	Los errores de dominio tienen tipo propio y se traducen a códigos del contrato (`no_encontrado`, `conflicto`, `validacion`) en el borde HTTP, en vez de excepciones genéricas o valores nulos.	`backend/src/modules/evidencias/domain/errores.ts:10`, `responderError` en `backend/src/infrastructure/http/validation.ts:17` y `docs/contrato-api.md`
No repetirse (DRY)	Cada tipo de la API se declara una sola vez, en `shared/`, y lo usan el backend y el cliente.	`shared/src/index.ts:1`; nota 004
Simplicidad (KISS y YAGNI)	Se demuestra por lo que no hay: sin ORM (los repositorios usan `pg` y SQL directo) y sin contenedor de inyección (los puertos se conectan a mano en una sola función). Un puerto existe solo donde hay un límite real con lo externo.	`backend/package.json` (dependencias) y `backend/src/composition-root.ts:87`; `AGENTS.md`
Pruebas limpias	Dominio y casos de uso se prueban rápido y sin infraestructura; los adaptadores contra una base real, nunca con imitaciones de ella.	`backend/src/modules/evidencias/evidencias.test.ts:29` (repositorios en memoria) y `backend/src/adaptadores-pg.test.ts:27` (PostgreSQL)

Dos principios estructurales completan los anteriores: alta cohesión, porque cada módulo es dueño de sus tablas y de sus reglas, y bajo acoplamiento, porque los módulos solo se comunican por su `index.ts` público y sin ciclos. `backend/src/arquitectura.test.ts` vigila dos reglas: que ningún módulo importe archivos internos de otro (línea 19) y que `composition-root.ts` no contenga SQL (línea 44). No comprueba la ausencia de ciclos ni que un repositorio lea solo sus tablas.

Alternativas descartadas

Alternativa	Por qué no
Microservicios	Resuelven escala y equipos independientes que aquí no existen, y cobran red, despliegues y consistencia entre bases. Con tres personas y una máquina local no aportan nada.
Arquitectura por capas clásica	La dependencia apunta a la base de datos, así que probar una regla exige una base, y los módulos terminan acoplados por las tablas.
MVC	Organiza la presentación, no el sistema; no dice dónde van las reglas de negocio ni dónde están los límites con lo externo.
Cliente-servidor con cliente en línea	Es el estilo del ejemplo del curso, pero exige conexión durante la operación, que es justo lo que los operarios no tienen.

Cliente

El cliente es una PWA que funciona primero sin conexión. Todas las acciones del operario se completan contra el almacenamiento local, y un servicio de sincronización aparte vacía una cola hacia la API. Usa los tipos y esquemas de `@plomada/shared` para hablar con el servidor, así el contrato es el mismo en ambos lados.

6. Diagrama de componentes

El backend es un monolito modular con arquitectura hexagonal en seis módulos (nota 001). La raíz de composición (`backend/src/composition-root.ts`) es el único lugar donde se instancian los adaptadores concretos y se inyectan en los casos de uso. No contiene SQL, y `backend/src/arquitectura.test.ts` lo comprueba.

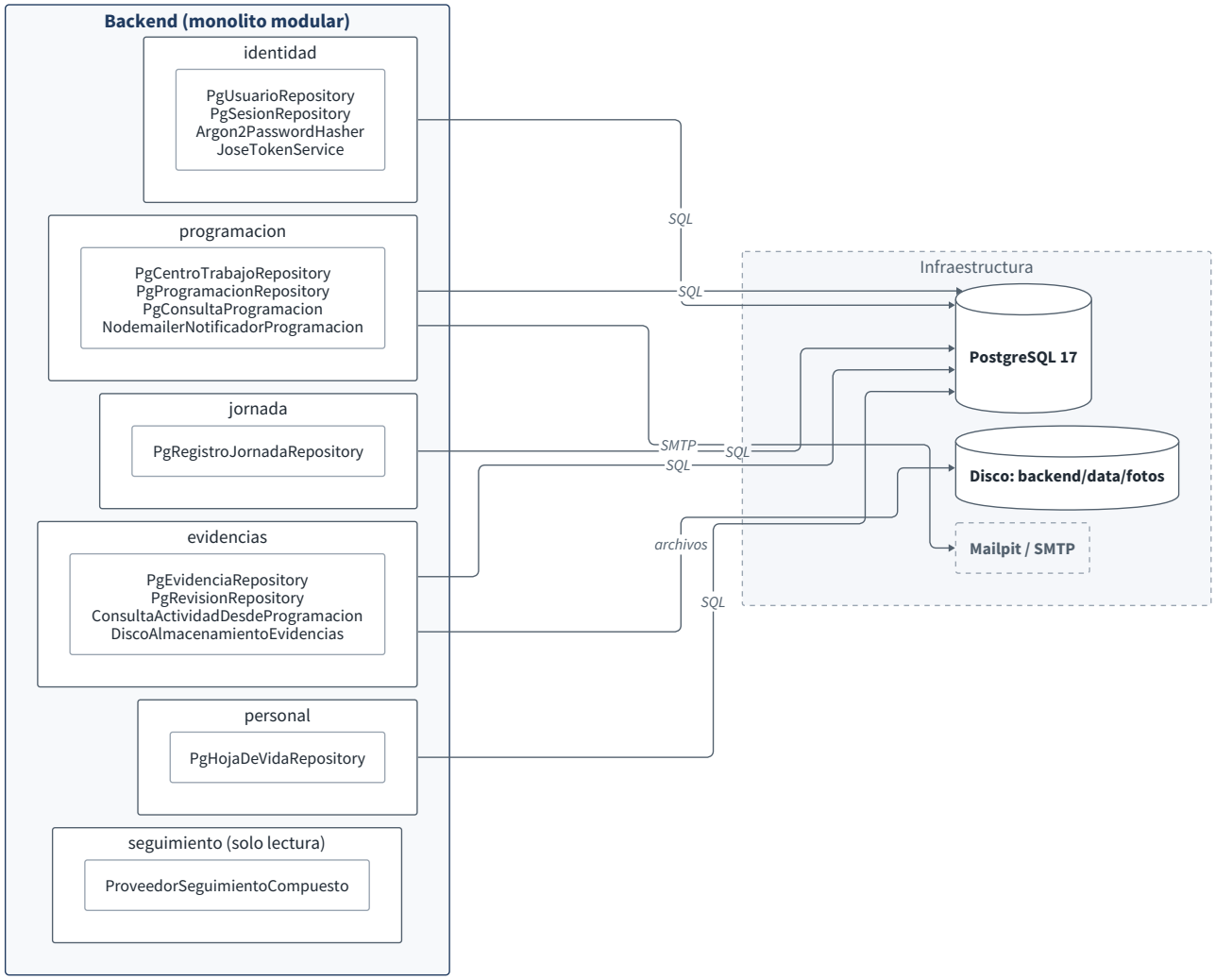


Figura 10. Diagrama de componentes

El segundo diagrama muestra lo que conecta la raíz de composición: los puertos que un módulo declara y otro usa, y qué implementación entrega cada uno.

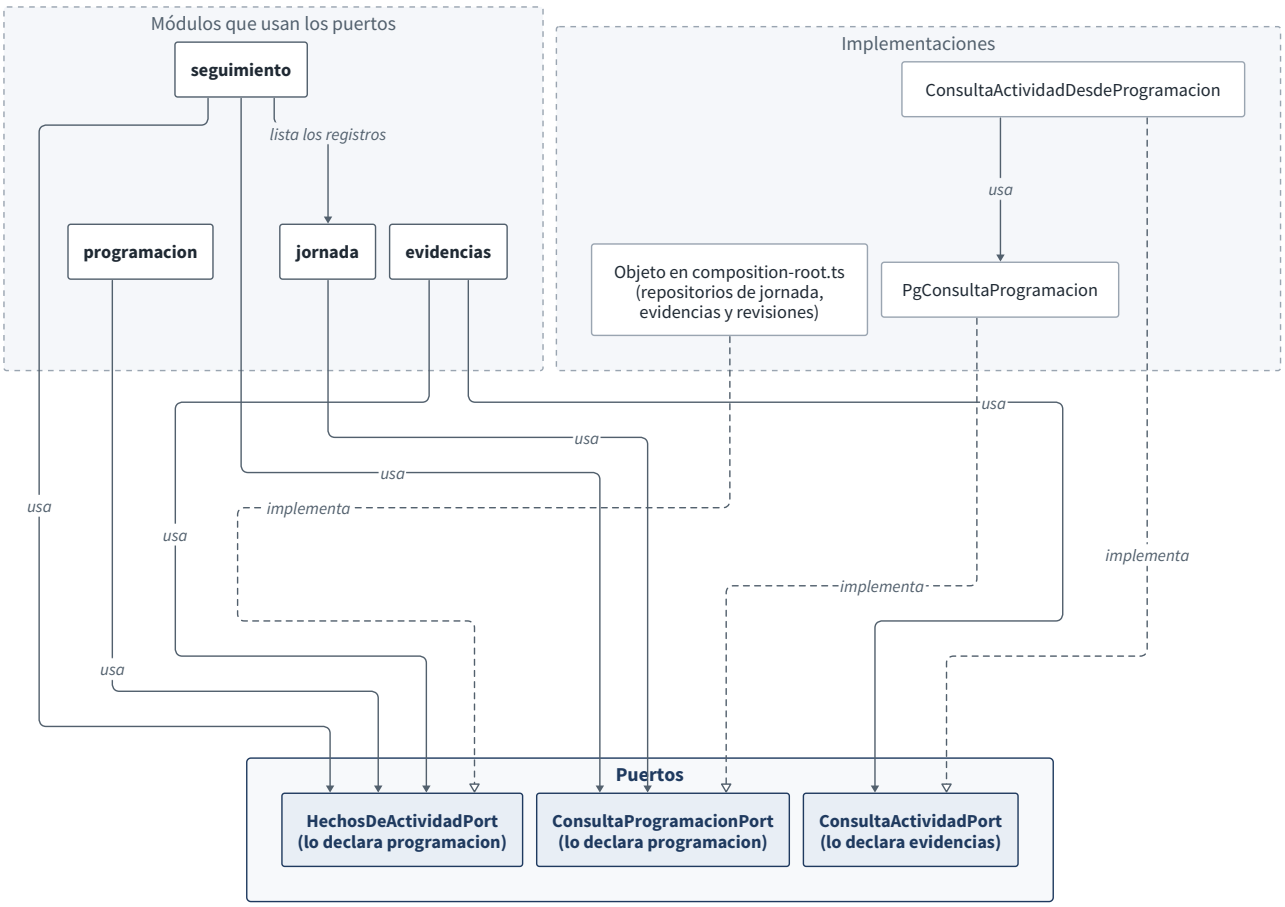


Figura 11. Puertos que cablea la raíz de composición

Cada módulo se usa solo por su `index.ts`:

- **`identidad`**: inicio de sesión, renovación de la sesión, usuarios autenticados y alta y listado de usuarios. Puertos `UsuarioRepositoryPort`, `SesionRepositoryPort`, `PasswordHasherPort` y `TokenServicePort`, con los adaptadores `PgUsuarioRepository`, `PgSesionRepository`, `Argon2PasswordHasher` y `JoseTokenService`.
- **`programacion`**: centros de trabajo, programaciones semanales, publicación con envío de correo y programación del operario (`ObtenerMiProgramacionUseCase`); exporta la función pura `derivarEstado`. Puertos `CentroTrabajoRepositoryPort`, `ProgramacionRepositoryPort` y `NotificadorProgramacionPort` (adaptador `NodemailerNotificadorProgramacion`); declara además `ConsultaProgramacionPort` (adaptador `PgConsultaProgramacion`) y `HechosDeActividadPort` para los demás módulos.
- **`jornada`**: registro de marcas de horario con UUID del cliente (`RegistrarJornadaUseCase`) y su consulta (`ListarRegistrosJornadaUseCase`). Puerto `RegistroJornadaRepositoryPort`; usa `ConsultaProgramacionPort` para comprobar el centro y la actividad.
- **`evidencias`**: subida de fotos con UUID del cliente (`SubirEvidenciaUseCase`), descarga de la foto (`ObtenerFotoEvidenciaUseCase`), cola de actividades del supervisor y revisión (`RevisarActividadUseCase`). Puertos `EvidenciaRepositoryPort`, `RevisionRepositoryPort`, `AlmacenamientoEvidenciasPort` (adaptador `DiscoAlmacenamientoEvidencias`) y `ConsultaActividadPort` (adaptador `ConsultaActividadDesdeProgramacion`).
- **`personal`**: hojas de vida de los operarios (`ObtenerHojaDeVidaUseCase`, `GuardarHojaDeVidaUseCase`). Puerto `HojaDeVidaRepositoryPort`, adaptador `PgHojaDeVidaRepository`.
- **`seguimiento`**: solo lectura. `ObtenerSeguimientoDiaUseCase` arma el día con `ProveedorSeguimientoCompuesto`, que usa `ConsultaProgramacionPort`, `HechosDeActividadPort` y el listado de `jornada`.

7. Diagrama entidad relación

El modelo relacional refleja los límites de cada módulo con claves foráneas por identificador (UUID). Cada repositorio lee y escribe solo las tablas de su módulo. Se comprobó abriendo los nueve archivos `Pg*` de `infrastructure/persistence`: `PgConsultaProgramacion` lee solo `programaciones_semanales`, `asignaciones`, `actividades` y `centros_trabajo`, que son de `programacion`, y `ProveedorSeguimientoCompuesto` no ejecuta SQL, porque obtiene todo por puertos. Fuera de los repositorios solo ejecutan SQL el migrador y la semilla. Lo que impone una prueba es más estrecho: que la raíz de composición no contiene SQL y que ningún módulo importa archivos internos de otro (`backend/src/arquitectura.test.ts`).

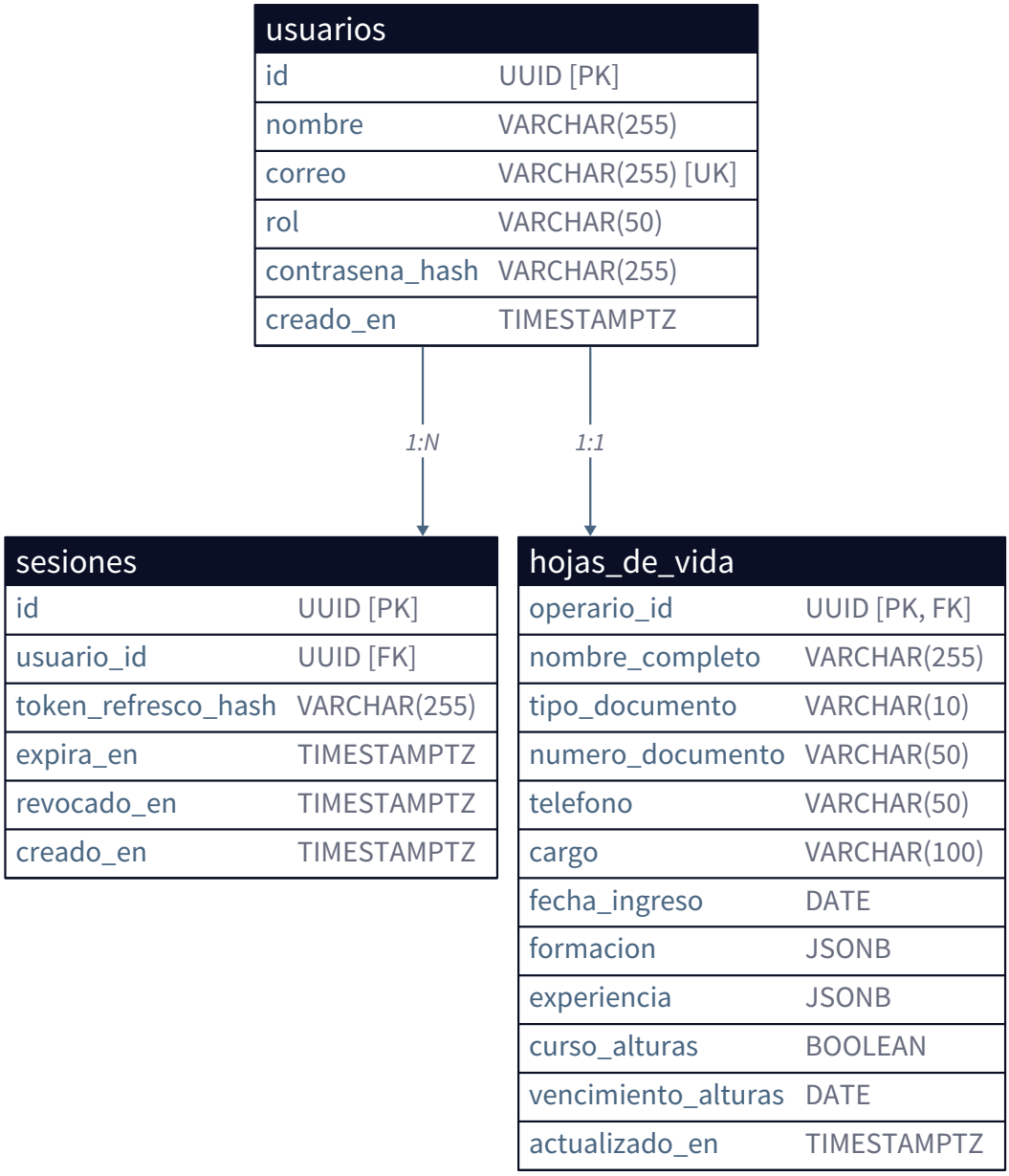


Figura 12. Diagrama entidad relación: usuarios, sesiones y hojas de vida

El segundo diagrama muestra las tablas de `programacion`; `usuarios` aparece solo como destino de la clave foránea de las asignaciones.

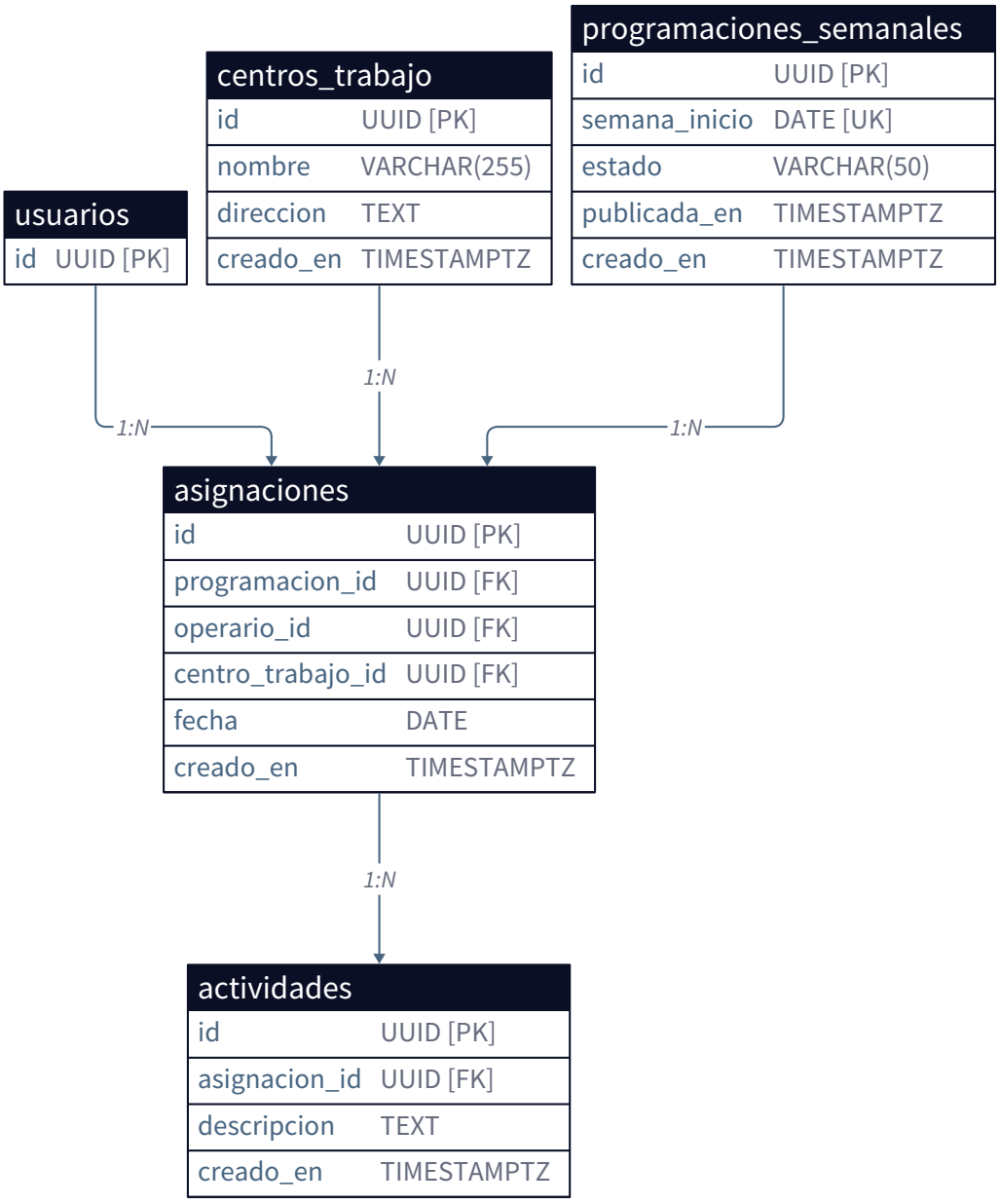


Figura 13. Diagrama entidad relación: programación

El tercer diagrama muestra las tablas de `jornada`, `evidencias` y las revisiones; `usuarios`, `centros_trabajo` y `actividades` aparecen solo como destino de sus claves foráneas.

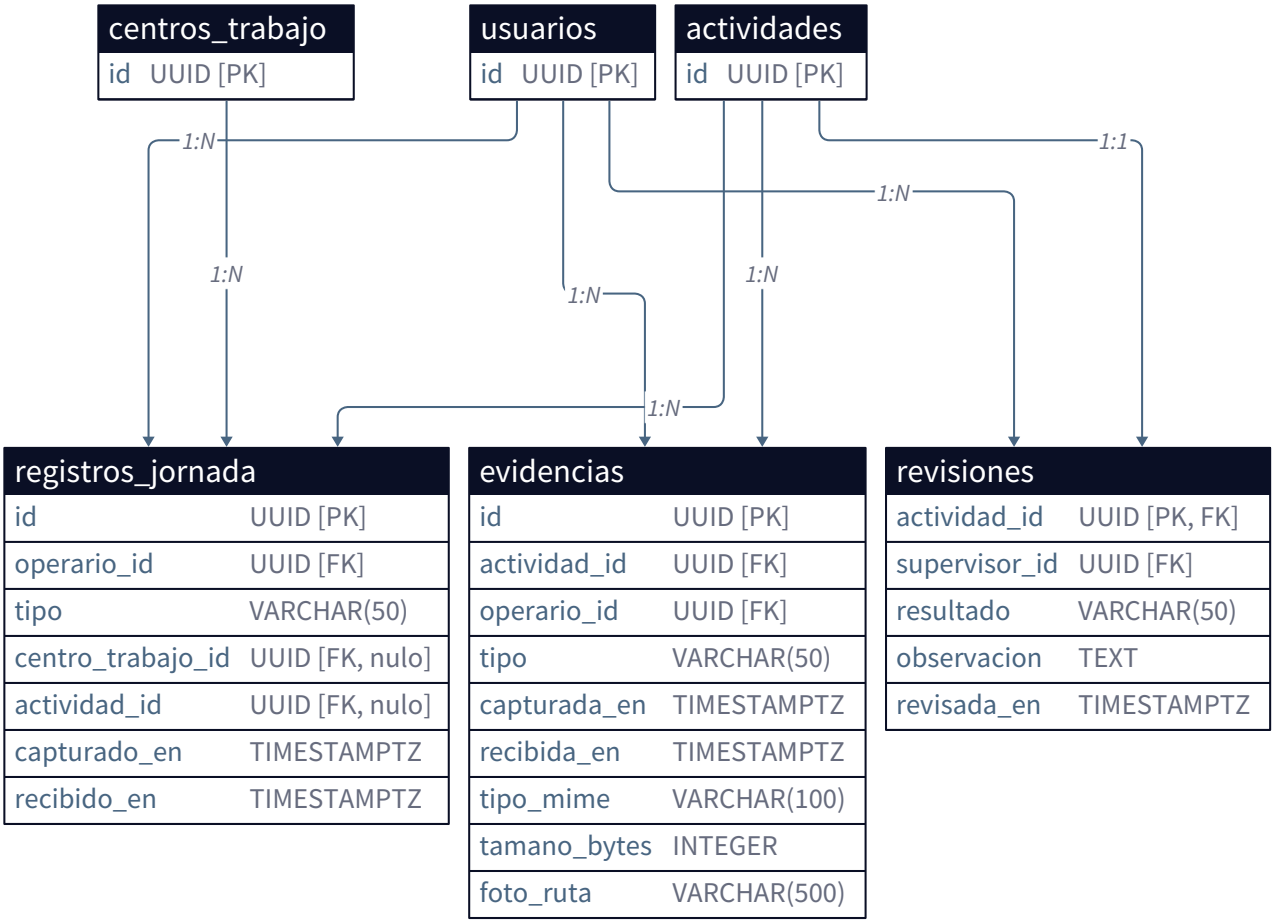


Figura 14. Diagrama entidad relación: jornada, evidencias y revisiones

Propiedades del modelo:

- **Inmutabilidad de los hechos**: `registros_jornada` y `evidencias` guardan lo que capturan los operarios. Ninguna consulta del código hace `UPDATE` ni `DELETE` sobre ellas: las inserciones usan `ON CONFLICT (id) DO NOTHING` y `evidencias.actividad_id` es `ON DELETE RESTRICT`. Es una garantía de la aplicación, no de la base, que no impide modificar esas filas a quien tenga acceso directo. La semilla, que no corre en producción, las vacía con `TRUNCATE`.
- **Dos horas por hecho**: `capturado_en` y `recibido_en` en `registros_jornada`, y `capturada_en` y `recibida_en` en `evidencias`. La primera es la del dispositivo y la segunda la del servidor.
- **Estado derivado**: la tabla `actividades` no tiene columna de estado. El estado (`pendiente`, `en_curso`, `finalizada`, `aprobada`, `rechazada`) lo calcula la función pura `derivarEstado` a partir de las evidencias, las marcas de inicio y fin y la revisión.
- **Una decisión por actividad**: la clave primaria de `revisiones` es `actividad_id`, de modo que cada actividad admite una sola revisión. Que un rechazo traiga observación lo impone el esquema zod `RevisionSolicitud` (`shared/src/evidencias.ts:40`), no la tabla, cuya columna `observacion` admite nulos.
- **Fotos fuera de la base**: `evidencias` guarda los metadatos y la ruta relativa en `foto_ruta`. Los bytes quedan en disco, detrás de `AlmacenamientoEvidenciasPort`.

8. Justificación de la base de datos

Se elige **PostgreSQL 17**. La alternativa no relacional que se evaluó es una base documental como MongoDB, que es la que un equipo con JavaScript consideraría primero. La comparación se hace sobre los cuatro puntos que pesan en Plomada:

Criterio	PostgreSQL	Base documental (MongoDB)
Relaciones	Las entidades se relacionan por clave foránea: usuario, asignación, actividad, evidencia y revisión forman una cadena. Una consulta las une sin duplicar datos.	Se modela anidando o duplicando. Una actividad con sus evidencias cabe en un documento, pero el supervisor y el operario también las consultan por separado, y lo anidado se desincroniza.
Integridad	El motor impide una evidencia sin actividad, una revisión duplicada por actividad o un rol inválido (`FOREIGN KEY` , `PRIMARY KEY` , `CHECK`). Las pruebas de adaptadores lo verifican.	La integridad referencial pasa a la aplicación; un error de código deja datos huérfanos sin que nadie lo note.
Consultas del supervisor	La cola de revisión cruza actividad, asignación, centro, operario y conteo de evidencias en una sola consulta con `JOIN` e índices sobre las claves. El estado derivado necesita esos cruces.	Los cruces entre colecciones se hacen con `$lookup` o en la aplicación, con menos ayuda del planificador.
Transacciones	Un registro y su efecto se escriben de forma atómica, y `INSERT ... ON CONFLICT DO NOTHING` da la idempotencia por UUID que exige la nota 005 en una sola sentencia.	Las transacciones multidocumento existen, pero el modelo está pensado para evitarlas; la idempotencia se resuelve con índices únicos y cuidado.

Dos argumentos más inclinan la decisión. Los datos de Plomada son estructurados y estables: las columnas se conocen y cambian poco, así que la flexibilidad de esquema de un documento no se aprovecha. Y las hojas de vida del módulo `personal` sí tienen forma variable (formación y experiencia); se guardan en una columna `JSONB` , de modo que el motor relacional absorbe ese caso sin sumar un segundo motor.

Las fotos no van en la base: se guardan en disco detrás del puerto `AlmacenamientoEvidencias` y en PostgreSQL solo quedan los metadatos, para no inflar la base ni sus respaldos (nota 007).

Conclusión. El sistema es esencialmente relacional y exige integridad y consultas cruzadas, y eso es lo que PostgreSQL resuelve mejor. El costo es esquema rígido y migraciones SQL, que con tres integrantes y un modelo estable es aceptable (nota 012).

9. Diagrama de despliegue

No hay despliegue en un servidor: el taller permite un despliegue local y no hay recursos para otro. Todo corre en un portátil, y el diagrama describe ese entorno.

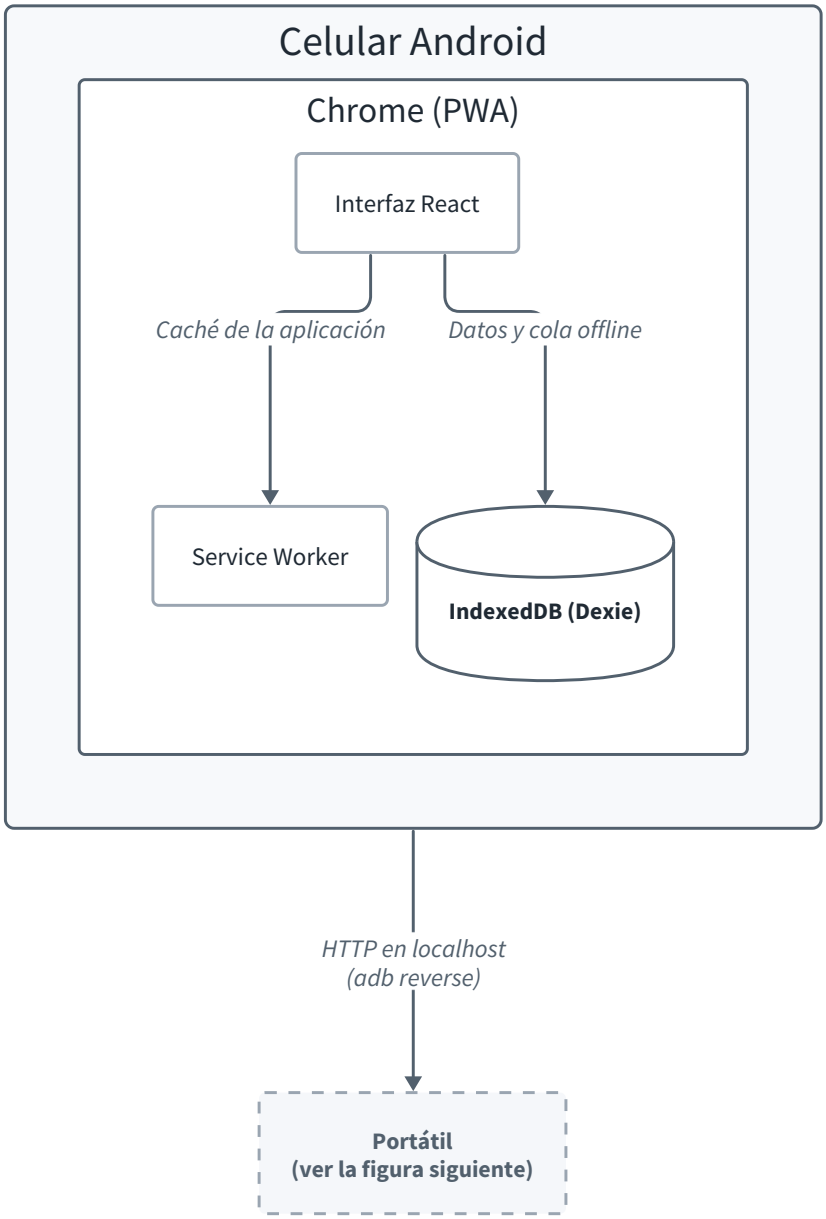


Figura 15. Diagrama de despliegue: el celular

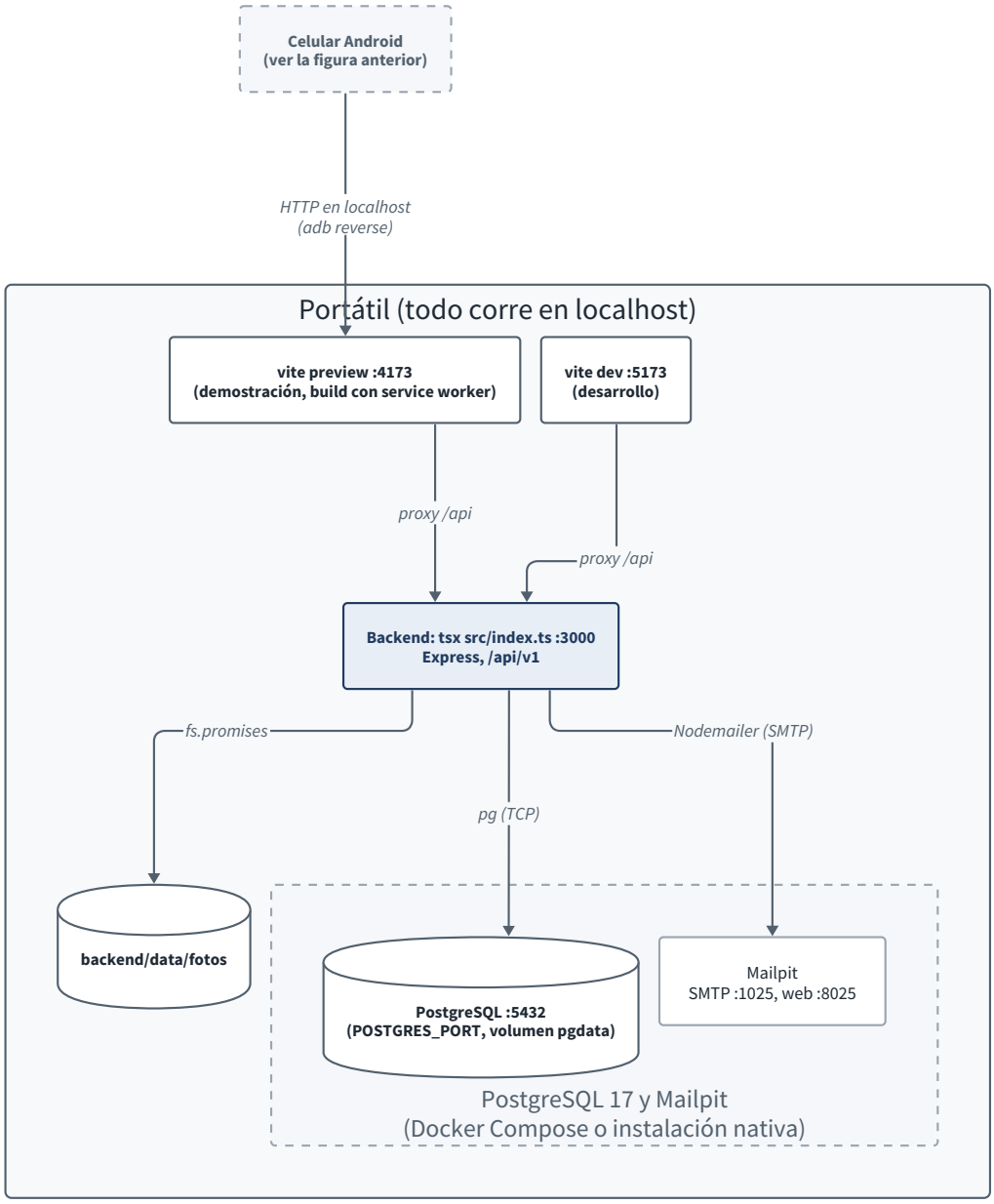


Figura 16. Diagrama de despliegue: el portátil

Aspectos del despliegue:

- **Infraestructura**: hay dos formas de levantarla. Con Docker Compose (`docker compose up -d`) arrancan PostgreSQL 17 (`postgres:17-alpine`) y Mailpit. Sin Docker se usa un PostgreSQL 17 nativo y el ejecutable de Mailpit, como describe la sección "Entorno sin Docker" del README.
- **Procesos**: el backend corre con `tsx` en el puerto 3000. La PWA se sirve con `vite preview` en el 4173 para la demostración, porque el service worker solo existe en el build, y con `vite dev` en el 5173 para desarrollo. Los dos reenvían `/api` al backend (`frontend/vite.config.ts`), así que el navegador usa un solo origen.
- **Datos**: PostgreSQL escucha en el 5432, configurable con `POSTGRES_PORT` , con el volumen `pgdata` y sin límite de memoria. Mailpit recibe SMTP en el 1025 y muestra su bandeja en el 8025. El backend guarda las fotos en `backend/data/fotos` (variable `FOTOS_DIR`).

- **Acceso**: todo va por HTTP en `localhost`, porque TLS queda fuera de v1 (ítem 1). El celular entra por `adb reverse` a `localhost`, que es un contexto seguro; `docs/demostracion.md` da los comandos y `docs/pruebas-celular.md` (sección 5) explica por qué. Por IP en HTTP no hay service worker ni almacenamiento persistente, y la prueba C-7 se hizo así (`docs/pruebas-celular.md`).

Inventario de recursos

Recurso	Tipo	Cantidad	Especificación / Ubicación	Propósito
Portátil	Equipo	1	Ejecuta el backend, los servidores de Vite, PostgreSQL y Mailpit	Entorno local de desarrollo y de demostración
Node.js	Entorno de ejecución	1	Versión 22 o superior (`engines` de `package.json`)	Backend con `tsx` y herramientas del cliente
pnpm	Gestor de paquetes	1	10.28.0 (`packageManager` de `package.json`)	Instalación y scripts del workspace
PostgreSQL	Base de datos	1	Versión 17: imagen `postgres:17-alpine` o instalación nativa. Puerto 5432, volumen `pgdata`	Datos de usuarios, programación, jornada y evidencias
Mailpit	Servidor SMTP de prueba	1	Imagen `axllent/mailpit` o ejecutable. SMTP en el 1025, web en el 8025	Recibir los correos de publicación de la programación
Directorio de fotos	Disco	1	`backend/data/fotos`. La proyección de 1,1 GB al día es un supuesto de volumen del ítem 1, no una medición	Guardar las fotos comprimidas de los operarios
Celular Android	Dispositivo	1	El de la prueba C-7, con Chrome. Modelo y versión por confirmar (`docs/pruebas-celular.md`)	Probar el cliente en un dispositivo real

10. Tecnologías

Estas son las tecnologías del proyecto y la razón de cada una.

Capa	Tecnología	Justificación
Lenguaje	TypeScript 7 (modo estricto), Node.js 22	Tipado estricto en todos los paquetes y módulos ECMAScript nativos.
Gestor de paquetes	pnpm 10 con workspaces y catálogos	El lockfile fija las versiones y los paquetes comparten dependencias sin duplicarlas en disco.
Contrato API	zod 4 en `@plomada/shared`	Los tipos y la validación son los mismos en el cliente y en el servidor.
Cliente web / PWA	React 19, Vite 8, Dexie.js, `vite-plugin-pwa`	Aplicación instalable que funciona primero sin conexión: guarda los datos en IndexedDB con tipos (nota 011) y comprime las fotos con Canvas.
Servidor HTTP	Express 5	Monolito modular con rutas REST en `/api/v1`.
Seguridad y tokens	`@node-rs/argon2`, `jose`	argon2id para las contraseñas; JWT para el token de acceso y un hash para el token de renovación.
Subida de archivos	`multer`	Recibe la foto en memoria, la entrega al almacenamiento en disco y limita el tamaño a 1,5 MB.
Base de datos	PostgreSQL 17 con cliente `pg`	Repositorios con SQL escrito a mano y transacciones manuales; migraciones SQL numeradas con un ejecutor propio (nota 012).
Correo	Nodemailer sobre SMTP (Mailpit en desarrollo)	Envía el aviso de la programación al publicarla, sin bloquear la respuesta.
Pruebas	Vitest 5	Pruebas de dominio con repositorios en memoria y pruebas de integración contra PostgreSQL real.
Infraestructura local	Docker Compose (opcional)	Levanta la base de datos y el correo en contenedores. No es obligatorio: también se pueden instalar sin Docker (README).
Documentación	Markdown, D2 y Playwright	Los diagramas se escriben como código y se generan en vectores; el documento imprimible sale de este mismo archivo.

11. Decisiones críticas

Cada decisión crítica cuenta con una nota estructurada en `docs/decisiones/` (problema, opciones y selección justificada):

001. Monolito modular con arquitectura hexagonal

Responsable: A. Estado: tomada.

Problema encontrado

Hace falta un estilo arquitectónico que sirva a unos 200 usuarios, con un cliente que trabaja sin conexión, que construyan tres personas en pocas semanas, que se despliegue en local y que permita demostrar y justificar principios de diseño con al menos dos módulos funcionando. La carga es baja; el riesgo real está en la integración entre personas y en que las reglas del negocio queden enredadas con la base de datos, el disco y el correo.

Opciones evaluadas

Microservicios, arquitectura por capas clásica, MVC, y monolito modular con arquitectura hexagonal dentro de cada módulo.

Los microservicios resuelven problemas de escala y de equipos independientes que este proyecto no tiene, y cobran su precio completo: red entre servicios, despliegues múltiples, consistencia entre bases y observabilidad. Con tres integrantes y una máquina local ese costo no compra nada. La arquitectura por capas clásica separa presentación, negocio y datos, pero la dependencia apunta hacia la base de datos, de modo que probar una regla exige una base, y con el tiempo los módulos se acoplan por las tablas. MVC es un patrón para organizar la capa de presentación, no una arquitectura de sistema, y no dice dónde van las reglas ni los límites con lo externo.

Opción seleccionada y por qué

Monolito modular, con cada módulo organizado en dominio, casos de uso y adaptadores, y con puertos que invierten la dependencia hacia lo externo. Es un solo despliegue, que es lo que el contexto pide. Cada módulo oculta sus detalles detrás de una interfaz pública, lo que da alta cohesión y bajo acoplamiento y deja la puerta abierta a extraer un módulo si algún día hiciera falta. Los puertos aíslan exactamente lo que cambia o estorba en las pruebas (PostgreSQL, el disco de las fotos, el servidor SMTP), de modo que el dominio y los casos de uso se prueban rápido con repositorios en memoria. La separación en módulos también sirve para el reparto del trabajo: B y C pueden avanzar sin pisarse.

Consecuencias

Cada módulo tiene más archivos que en un diseño plano. Para que esa ceremonia no se vuelva un fin en sí misma, la regla de AGENTS.md es no crear un puerto donde no haya un segundo adaptador ni una razón real para cambiar; los puertos existen solo en los límites con la base de datos, el almacenamiento de fotos, el correo y HTTP.

002. PWA que funciona primero sin conexión

Responsable: A. Estado: tomada.

Problema encontrado

Los operarios no tienen datos móviles y el conjunto cerrado no garantiza ni prohíbe internet. Aun así deben registrar horas y tomar tres fotos por actividad, y esa evidencia tiene que llegar completa y sin duplicados. Usan sus propios celulares, de gama baja o media, sin que la empresa pueda instalarles nada ni gestionarlos.

Opciones evaluadas

Una aplicación nativa para Android e iOS, una aplicación híbrida (Capacitor o React Native), una web que exige conexión y una PWA diseñada para trabajar sin conexión.

Una web que exige conexión incumple la restricción principal del problema. Una aplicación nativa o híbrida resuelve el trabajo sin conexión, pero obliga a publicar en tiendas, mantener dos plataformas y pedir a 150 personas que instalen y actualicen una aplicación en dispositivos que nadie controla. La PWA se instala desde el navegador, es una sola base de código y alcanza para lo que el problema necesita: guardar datos en IndexedDB, cachear la aplicación con un service worker y sincronizar con una cola.

Opción seleccionada y por qué

PWA que funciona primero sin conexión. Toda acción del operario se completa contra el almacenamiento local y responde de inmediato, con o sin red; la sincronización es un proceso aparte que drena una cola cuando hay conexión. Es la única opción que cumple la restricción sin cargar a la empresa con distribución y mantenimiento de aplicaciones.

Consecuencias

El service worker y `crypto.randomUUID()` solo existen en un contexto seguro, es decir, HTTPS o `localhost`. Para el demo se usa el navegador del portátil con la emulación de móvil y el modo sin conexión de las herramientas de desarrollo; probar en un celular real se hace con `adb reverse` hacia `localhost`, que es un contexto seguro (`docs/pruebas-celular.md`; corregido el 7 de noviembre de 2026, cuando la nota decía HTTPS local con certificado o túnel). La Background Sync API no es utilizable porque iOS no la soporta, así que la sincronización se dispara al abrir la aplicación, al volver la conexión y con un botón manual. Las cuotas de almacenamiento de los teléfonos de gama baja obligan a comprimir las fotos antes de encolarlas (nota 007).

003. PERN con TypeScript, workspace de pnpm y paquete shared

Responsable: A. Estado: tomada.

Problema encontrado

El stack es PERN por decisión del grupo, y falta definir cómo se organiza el código para que tres personas trabajen en paralelo sin que el backend y el frontend se desalineen en silencio, y sin que dos integrantes con poca práctica en Git tengan que pelear con herramientas.

Opciones evaluadas

JavaScript o TypeScript. Repositorios separados o un monorepo. npm, yarn o pnpm. Declarar los tipos de la API en cada lado o compartirlos.

JavaScript tiene menos fricción inicial, pero deja los puertos hexagonales como una convención que existe solo en la cabeza y en el documento, y deja el contrato entre B y C a merced de la disciplina. Repositorios separados obligan a publicar el contrato como paquete y a coordinar versiones, lo cual es mucho para un taller. Entre los gestores de paquetes, pnpm ofrece workspaces y un catálogo de versiones únicas, y su archivo `packageManager` fija la misma versión para todo el equipo.

Opción seleccionada y por qué

TypeScript en modo estricto en todo el código, en un monorepo de pnpm con tres paquetes (`shared`, `backend` y `frontend`) y las versiones de las dependencias comunes centralizadas en el catálogo de `pnpm-workspace.yaml`. El paquete `shared` contiene los esquemas del contrato, de modo que un cambio en un campo hace fallar la compilación del otro paquete en lugar de fallar en la integración. Los puertos son interfaces verificables por el compilador, y el repositorio en memoria y el de PostgreSQL deben implementar la misma. Con IA en el equipo, el

compilador además actúa como red de seguridad contra código plausible pero incorrecto. PostgreSQL es la base elegida por la fuerte relación entre operario, asignación, actividad y evidencia; B desarrolla la justificación en el ítem 8 del documento.

Consecuencias

El esqueleto, que ya está montado, absorbe la configuración inicial, y el equipo paga la curva de TypeScript. Las versiones actuales (TypeScript 7, Vite 8, Vitest 5) son mayores recientes: si alguna herramienta que B o C quieran sumar falla con ellas, la salida es anclar una versión anterior en el catálogo mediante un PR de A, no resolverlo con parches locales.

004. Contrato REST expresado como esquemas zod compartidos

Responsable: A. Estado: tomada.

Problema encontrado

B implementa la API y C construye el cliente al mismo tiempo. Si el acuerdo entre ambos vive solo en un documento, el desacuerdo sobre un campo o un código de error aparece en la integración, que es cuando más cuesta corregirlo.

Opciones evaluadas

Un documento en Markdown o una especificación OpenAPI escrita a mano. Una especificación OpenAPI generada desde el código. Tipos de TypeScript compartidos sin validación en ejecución. Esquemas zod en un paquete compartido junto con un documento en prosa. En cuanto al estilo de la API, REST con JSON frente a GraphQL o tRPC.

Un documento o un OpenAPI a mano describen el contrato pero no obligan a nadie a cumplirlo, y se desactualizan sin aviso. Generar OpenAPI exige herramientas adicionales y deja dos modelos de datos que mantener. Los tipos sin validación aseguran la compilación pero no lo que realmente llega por la red, y los datos que vienen del celular, del disco o de una variable de entorno hay que validarlos igual. GraphQL y tRPC no aportan nada a unas 25 rutas, y la subida de fotos en multipart y los `PUT` idempotentes del cliente sin conexión encajan de forma natural en REST.

Opción seleccionada y por qué

Esquemas zod en `@plomada/shared`, un registro de rutas con método, acceso y prioridad en `shared/src/contrato.ts`, y un documento en prosa en `docs/contrato-api.md`. De una sola definición salen el tipo de TypeScript y la validación en ejecución, y el compilador falla en el otro paquete cuando algo cambia. Una prueba de `shared` verifica que cada ruta del código aparezca en el documento, así que los dos no pueden separarse sin que el CI lo note.

Consecuencias

Zod viaja en el cliente, con un costo de peso aceptable. Todo cambio de contrato pasa por un PR que A aprueba, lo cual centraliza un punto de decisión; a cambio, el compromiso es revisarlo el mismo día para que no frene a nadie.

005. Escrituras idempotentes con UUID del cliente, dos horas por evento y estado derivado

Responsable: A. Estado: tomada.

Problema encontrado

Las fotos y los registros nacen sin conexión y llegan al servidor horas después. Un corte de red a mitad de un envío deja al cliente sin saber si el servidor lo recibió, y repetirlo no puede producir una segunda foto. Además, el reloj de un celular personal no es fiable, y el control de horario del operario depende de saber cuándo ocurrió cada hecho.

Opciones evaluadas

Para el envío, un endpoint de sincronización por lotes, un `POST` por recurso con identificador asignado por el servidor, un `PUT` por recurso con UUID generado en el cliente, o una biblioteca de sincronización con replicación (PouchDB, RxDB). Para el tiempo, fiarse de la hora del celular, fiarse de la hora del servidor o guardar ambas.

El lote falla completo si una sola foto pesada se corta, y obliga al servidor a manejar éxito parcial. El `POST` con id del servidor tiene justo el problema que hay que evitar: si la respuesta se pierde, el cliente reintenta y se crea un duplicado. Las bibliotecas de replicación resuelven más de lo necesario e imponen su propia base de datos en ambos lados.

Opción seleccionada y por qué

`PUT` individual sobre un UUID que el cliente genera al capturar. El servidor responde 201 la primera vez, 200 si ya lo tiene con el mismo contenido y `conflicto` si el contenido difiere, de modo que reintentar siempre es seguro y el servidor no guarda estado de sesiones de sincronización. Un fallo afecta a un solo elemento, y el cliente puede insistir en ese sin repetir el resto.

Cada hecho guarda dos horas, `capturadoEn` (la del celular, en el instante del evento) y `recibidoEn` (la del servidor), las dos en UTC, sin corregir ninguna. Mostrar las dos al supervisor es más honesto que adivinar cuál es la buena.

El estado de una actividad no se almacena: se deriva de los hechos registrados con reglas fijas (ver `docs/contrato-api.md`). Así el resultado no depende del orden de llegada, y una foto `despues` que llega antes que la `antes` produce el mismo estado final.

Consecuencias

El cliente drena la cola de a un elemento, lo cual es más lento que un lote en un enlace bueno pero mucho más robusto en uno malo. Un reloj muy desajustado produce un `capturadoEn` dudoso que el sistema no detecta ni corrige; es una limitación asumida y documentada.

006. Token de acceso corto y refresh rotativo

Responsable: A. Estado: tomada.

Problema encontrado

El enunciado pide definir protocolos de autenticación y cifrado. La dificultad es que un operario puede pasar horas sin conexión y sincronizar después, con una sesión que en ese momento ya debería haber vencido, sin que por eso haya que dejar un token de larga vida expuesto en un celular personal.

Opciones evaluadas

Un JWT de vida larga, una sesión con cookie validada por el servidor, un JWT de acceso corto con refresh token opaco rotativo, y un proveedor externo de identidad (inicio de sesión único con el correo institucional).

Un JWT de varios días no se puede revocar y, si se roba el celular, da acceso durante todo ese tiempo. La cookie de sesión exige que el servidor valide en cada petición y encaja mal con una cola que sincroniza horas después. Un proveedor externo depende de qué plataforma de correo use la empresa, un dato que el enunciado no da, y excede el alcance del taller.

Opción seleccionada y por qué

Token de acceso JWT de 15 minutos y refresh token opaco de 7 días que rota en cada uso y se guarda con hash en la base de datos. Las contraseñas se almacenan con argon2id. El rol viaja en el token: el rol se comprueba en el router y la propiedad del recurso en la capa de aplicación (nota 014; la nota decía antes que todo se decidía en la capa de aplicación). En producción todo viaja por TLS. Antes de sincronizar, el cliente renueva la sesión con el refresh token; si eso falla con 401, el operario vuelve a iniciar sesión y la cola se conserva. Así el daño de un token robado queda acotado y la sesión se puede revocar, y aun así la cola funciona después de horas sin red.

Consecuencias

B implementa una tabla de refresh tokens y la rotación, lo que agrega trabajo al módulo `identidad`. Si ese costo resulta alto, B propone una simplificación mediante un PR a esta nota y al contrato, por ejemplo un refresh sin rotación, y nunca la aplica en silencio. Las fotos no se cifran a nivel de aplicación; se confía en el cifrado de disco del sistema operativo, y esa limitación queda en el documento.

007. Subida multipart, disco local detrás de un puerto y compresión en el cliente

Responsable: A. Estado: tomada.

Problema encontrado

Unos 150 operarios toman tres fotos por actividad desde teléfonos de gama baja, con enlaces que pueden ser malos o intermitentes. Hay que decidir cómo viajan las fotos, dónde se guardan y cuánto pesan, sin romper el despliegue local ni los límites de almacenamiento del celular.

Opciones evaluadas

Para el transporte, la imagen en base64 dentro de un JSON o un multipart hacia el backend. Para el almacenamiento, un servicio de objetos en la nube con URL prefirmada, columnas `bytea` en PostgreSQL o archivos en el disco del servidor.

Base64 aumenta el peso en un tercio y obliga a cargar la imagen entera en memoria. Un servicio en la nube exige una cuenta y conexión del servidor, y contradice el despliegue local. Guardar las imágenes en PostgreSQL infla la base y sus respaldos. El disco local es lo más simple que funciona.

Opción seleccionada y por qué

Multipart hacia el backend, con el archivo guardado en disco local (variable `FOTOS_DIR`) detrás de un puerto `AlmacenamientoEvidencias`, y solo los metadatos (ruta, tamaño, tipo) en PostgreSQL. El cliente comprime antes de encolar, a unos 400 KB y un lado mayor de 1600 px, en JPEG o WebP; el servidor rechaza todo archivo de más de 1.5 MB. Los valores exactos están en `LIMITES_EVIDENCIA`, para que ambos lados usen los mismos. Pasar por un puerto significa que pasar a almacenamiento en la nube más adelante es cambiar un adaptador y no tocar el dominio, y es el caso real de segundo adaptador que justifica ese puerto.

Consecuencias

El respaldo del directorio de fotos debe coordinarse con el de la base de datos. Con los supuestos del ítem 1 del documento, el volumen ronda 1 GB por día de trabajo, así que el disco del servidor es un recurso a vigilar. Las fotos no se cifran a nivel de aplicación (nota 006).

008. Alcance mínimo vertical y prioridades P0 a P2

Responsable: A. Estado: tomada.

Problema encontrado

El enunciado describe un sistema amplio (programación, jornada, evidencias, hojas de vida, seguimiento en tiempo real, correo) y el taller pide poner en funcionamiento al menos dos módulos, con tres personas y poco tiempo. Construir de más pone en riesgo lo esencial, y recortar sin criterio deja huecos que bloquean a otro integrante.

Opciones evaluadas

Construir todo el enunciado con poca profundidad, recortar módulos completos, o construir un recorrido vertical mínimo que atraviese el cliente, la API y la base de datos, y diseñar el resto sin implementarlo.

Opción seleccionada y por qué

Recorrido vertical mínimo. Es P0 lo necesario para la demostración: inicio y renovación de sesión, crear y publicar una programación con su correo, la programación del operario, la subida de evidencias con sincronización sin conexión, y la revisión del supervisor. Es P1 lo que completa el flujo: registros de jornada, usuarios y centros, y listados. Es P2 lo que solo se implementa si sobra tiempo: hojas de vida y seguimiento. Todo P1 y P2 queda en el contrato para que nadie tenga que adivinar su forma.

Quedan fuera de v1 y se documentan como limitaciones: la interfaz web de administración (excepción aprobada en la nota 013), la edición de una programación ya publicada, el reenvío de evidencia tras un rechazo, el historial de revisiones, los reportes y las notificaciones push. Como no hay interfaz de administración, los datos del demo (un administrador, un supervisor, tres operarios, dos centros y una programación) los crea un script de semilla del backend, y la programación se publica llamando a la API. Eso es honesto y suficiente para mostrar los principios que el taller evalúa.

Consecuencias

En la sustentación el administrador aparece como llamadas a la API y no como una pantalla, y hay que decirlo desde el principio. A cambio, el esfuerzo de C se concentra en lo que más pesa: la PWA del operario y la sincronización.

Revisión posterior

El código integrado incluye un panel web de administración, el seguimiento diario (CU-08) y el expediente laboral con el curso de alturas (CU-13). Nada de eso estaba en el alcance de esta nota: la interfaz de administración figuraba como excluida de v1 y el seguimiento y las hojas de vida como P2. Esta nota no se amplía por haberse construido. El panel lo evalúa y lo aprueba como excepción la nota 013, con su razón y sus límites; lo demás sigue siendo P2 y no sustituye a ningún P0 pendiente.

009. Seguimiento por polling

Responsable: A. Estado: tomada.

Problema encontrado

El enunciado pide control y seguimiento "en momento real" del trabajo de cada operario. Pero los operarios no tienen datos móviles, de modo que lo que el servidor sabe llega cuando sus celulares logran sincronizar, por lotes y con retraso. El tiempo real que se puede ofrecer está acotado por esa sincronización.

Opciones evaluadas

WebSocket, Server-Sent Events, polling periódico y prescindir del seguimiento.

Opción seleccionada y por qué

Polling de `GET /seguimiento/dia` cada 10 a 15 segundos desde la vista del supervisor. Un canal de empuje no mejora la frescura real, porque el dato no existe en el servidor antes de que el celular lo sincronice. El polling no necesita infraestructura adicional ni estado de conexiones, y se recupera solo de cualquier corte. La carga es trivial: unos 30 supervisores consultando cada 12 segundos son alrededor de 2.5 peticiones por segundo.

Consecuencias

"En momento real" significa, en este sistema, casi en tiempo real respecto de lo ya sincronizado, y así se declara en las limitaciones del documento. Si el cliente exigiera empuje, Server-Sent Events es el siguiente paso y reemplaza el polling sin cambiar el modelo de datos.

010. Grafo de dependencias entre módulos sin ciclos

Responsable: A. Estado: tomada.

Problema encontrado

Las reglas del sistema obligan a los módulos a conocerse. `evidencias` y `jornada` necesitan saber si una actividad existe y a quién está asignada, y eso pertenece a `programacion`. Pero `programacion` también debe devolver el estado de cada actividad, y ese estado se deriva de hechos que registran `evidencias` y `jornada`. Si cada módulo importa al otro, aparece un ciclo, y un ciclo entre módulos es el acoplamiento que el estilo modular existe para evitar. Si B y C descubren esto a mitad del desarrollo, alguien tendrá que rehacer trabajo.

Opciones evaluadas

Dejar que los módulos se importen libremente. Mover la derivación del estado a un módulo aparte que lea de todos. Hacer que `programacion` declare un puerto para lo que necesita y que la raíz de composición lo implemente. Comunicar los módulos mediante eventos de dominio.

Importarse libremente produce el ciclo. Un módulo aparte para derivar estados traslada a `seguimiento`, que es P2, rutas que son P0. Los eventos de dominio resuelven el ciclo pero agregan un mecanismo (bus, orden de entrega, manejo de fallos) desproporcionado para un monolito de seis módulos.

Opción seleccionada y por qué

Un grafo de dependencias sin ciclos en el que cada flecha significa "importa la interfaz pública de". `personal` y `programacion` dependen de `identidad`. `jornada` y `evidencias` dependen de `programacion`. `seguimiento`, que solo lee, depende de `programacion`, `jornada` y `evidencias`. Nadie depende de `seguimiento`.

Cuando un módulo necesita algo de otro que ya depende de él, declara un puerto con lo que necesita y la raíz de composición, que es el único archivo autorizado a conocer todos los módulos, lo implementa llamando a la interfaz pública del otro. Así `programacion` declara el puerto `HechosDeActividad` para armar el estado de las actividades que devuelve, y la raíz de composición lo implementa con lo que expongan `evidencias` y `jornada`. La regla que deriva el estado es una función pura del dominio de `programacion`, expuesta en su interfaz pública, y `evidencias` la usa para las rutas `/actividades`. Es inversión de dependencias aplicada a un caso real, sin mecanismos nuevos.

Consecuencias

La raíz de composición hace de pegamento entre módulos y crecerá un poco; debe seguir siendo cableado, sin reglas de negocio. Si B encuentra otro ciclo, aplica la misma salida y lo anota aquí en un PR; no lo resuelve con un import cruzado.

Al 7 de noviembre de 2026 el código no necesitó dos de las aristas previstas. `personal` y `programacion` no importan `identidad`, porque los datos de usuarios llegan por la raíz de composición. `seguimiento` no importa `evidencias`, porque lee los hechos de las actividades por `HechosDeActividadPort`, que la raíz implementa; solo importa `jornada` y `programacion`.

011. Almacenamiento local en el cliente PWA con Dexie.js

Responsable: C. Estado: tomada.

Problema encontrado

Los operarios de mantenimiento locativo trabajan en sótanos y áreas comunes de conjuntos residenciales donde la señal móvil es nula. La aplicación debe almacenar en el dispositivo personal (generalmente celulares de gama media o baja) la programación de la semana, los registros de jornada, las fotografías de evidencia y una cola de peticiones pendientes de subida. Se requiere una solución de almacenamiento local que admita objetos binarios (`Blob`) sin duplicar el consumo de memoria mediante base64, que ofrezca tipado estricto con TypeScript, esquemas declarativos versionados y transacciones atómicas para que el registro de una evidencia y su encolamiento ocurran de forma consistente.

Opciones evaluadas

1. `localStorage`: Limitado a ~5 MB por origen, acceso síncrono que bloquea el hilo principal de la interfaz gráfica y no soporta objetos binarios directamente, obligando a codificar fotos en base64 con una sobrecarga del 33% en tamaño y consumo excesivo de memoria RAM.
2. **IndexedDB nativo / `idb` minimalista**: IndexedDB es el estándar asíncrono del navegador para datos estructurados y binarios. Sin embargo, su API nativa basada en eventos DOM es propensa a errores, y el wrapper `idb` ofrece únicamente una capa de promesas sin validación de esquemas tipados ni consultas ergonómicas.
3. `Dexie.js`: Biblioteca madura y ligera sobre IndexedDB nativo. Provee definición declarativa de tablas e índices, soporte de esquemas versionados, tipado estricto en TypeScript con `EntityTable<T, K>`, almacenamiento directo de `Blob` y soporte de transacciones atómicas sobre varias tablas.

Opción seleccionada y por qué

Se selecciona **Dexie.js** para gestionar IndexedDB en el frontend.

Respeta el principio *offline-first* y la regla KISS de la arquitectura:

- Almacena las fotos comprimidas directamente como `Blob` en la tabla `evidenciasLocales`, evitando la penalización de CPU y memoria de base64 en dispositivos de bajos recursos.
- Facilita la creación atómica del recurso local y del elemento en la `colaSincronizacion`.
- Proporciona seguridad de tipos compartida con el paquete `shared`, evitando discrepancias de esquema.
- No introduce servidores ni dependencias pesadas en el cliente, manteniendo el bundle ligero.

Consecuencias

El almacenamiento local queda encapsulado en `frontend/src/infrastructure/storage/dexie-db.ts`. El motor de sincronización (`motor-sincronizacion.ts`) consume y actualiza las tablas de Dexie de manera predecible, permitiendo que la interfaz responda instantáneamente a las acciones del operario sin depender de la latencia ni de la disponibilidad de la red.

012. Migraciones SQL numeradas con un ejecutor propio

Responsable: B. Estado: tomada.

Problema encontrado

El valor por defecto de `docs/decisiones/README.md` para las migraciones era `node-pg-migrate`. El esquema del demo son seis archivos SQL planos y el ejecutor debe correr igual sobre `plomada` y `plomada_test`, sin que nadie edite la base a mano.

Opciones evaluadas

`node-pg-migrate`, que añade una dependencia, su propio formato de archivos y su configuración. Un ejecutor propio de un centenar de líneas sobre `pg`, que lee los `.sql` de `backend/db/migrations/` en orden y registra cada uno en la tabla `migraciones`.

Opción seleccionada y por qué

El ejecutor propio (`backend/src/infrastructure/database/migrator.ts`). Los archivos siguen siendo SQL numerado, como pide AGENTS.md, y cada migración se aplica una sola vez; las pruebas de adaptadores lo reutilizan para preparar `plomada_test`. Evita una dependencia que no aporta nada a este tamaño de esquema (KISS y YAGNI). Si el esquema crece o hacen falta migraciones de reversa, se reemplaza por `node-pg-migrate` sin tocar los archivos SQL.

Consecuencias

No hay migraciones de reversa: una corrección es una migración nueva. El ejecutor no calcula sumas de verificación, así que editar una migración ya aplicada no se detecta; la regla es no editarlas.

013. Panel web de administración como excepción al alcance v1

Responsable: A. Decisor: Brayan. Estado: tomada.

Problema encontrado

La nota 008 dejó la interfaz web de administración fuera de v1 y clasificó las hojas de vida y el seguimiento como P2. Prevé que el administrador aparezca en la demostración como llamadas a la API y datos de una semilla. El código integrado hace otra cosa: `frontend/src/components/` tiene pantallas para los cuatro roles (`AdminDashboard.tsx`, `AdministrativoDashboard.tsx`, `SupervisorDashboard.tsx` y `OperarioDashboard.tsx`). Las tres primeras suman unas 3600 líneas y cubren creación de usuarios y centros, programación y publicación, hojas de vida, seguimiento diario y registros de jornada. Llegaron en los commits cbb6b01 y 6ef0868, y en el segundo se agregó a la nota 008 una "Ampliación de alcance" que justificaba lo construido, sin pasar por una decisión previa. Además, ese commit cambió dos esquemas de `shared/`: `HojaDeVida` ganó `cursoAlturas` y `vencimientoAlturas` (con la migración `006_alturas_hoja_de_vida.sql`), y `UsuarioCrear` ganó validaciones de nombre, contraseña y dominio de correo. El alcance escrito y el alcance real divergen, y hay que decidir cuál de los dos se corrige.

Dos hechos pesan en la decisión. El enunciado del taller pide los módulos de hojas de vida, el seguimiento en momento real y el control de horario de entrada y salida; no pide una interfaz de administración, pero esos tres módulos solo se pueden mostrar con una pantalla o con llamadas a la API. Y la prueba de ejecución del QA ejerció los endpoints de estos flujos, no las pantallas: la vista del supervisor y los paneles de administrativo y administrador figuran como no ejecutados en su informe.

Opciones evaluadas

1. Retirar el panel y volver al alcance de la nota 008. Descartada: el plan de correcciones congela el código y no contempla retirar pantallas, y separar lo que sobra de las pantallas que sí sirven sería trabajo no planeado sobre archivos de más de mil líneas.
2. Conservarlo sin registrarlo. Descartada: deja una nota 008 que no describe el sistema, y quien presente no podría explicar de dónde salió ese alcance.
3. Conservarlo y aprobarlo como excepción acotada, con una nota que diga qué cubre y qué no.

Opción seleccionada y por qué

Opción 3. Se conserva el panel y se aprueba como excepción al alcance v1, por dos razones. Primera, el enunciado pide hojas de vida, seguimiento y control de horario, y el panel es la forma en que esos módulos se ven funcionar. Segunda, el panel usa solo rutas que ya están en `shared/src/contrato.ts`: el contrato no ganó ninguna ruta con él.

También se aprueba, con efecto retroactivo, el cambio de esquemas del commit 6ef0868, salvo la restricción de dominio del correo, que se revierte en la nota 015. El criterio es que un cambio de contrato sin PR previo se acepta después solo si es compatible, está cubierto por pruebas y queda documentado. Los campos de alturas son opcionales, tienen la migración 006 y una prueba de integración en el backend, así que no rompen a quien no los envíe. Las validaciones de `UsuarioCrear` (nombre de al menos 3 caracteres, contraseña inicial de al menos 8 que no sean espacios) endurecen la entrada, pero sus únicos consumidores, el servidor y el panel, ya las aplican, y las cubren las pruebas de `shared`. Quedan en el contrato y en `docs/contrato-api.md`. A partir de ahora, un cambio de contrato sin PR previo aprobado por A no se acepta de nuevo.

La aprobación tiene tres límites. No cambia las prioridades: la PWA del operario y la sincronización sin conexión siguen siendo lo que decide la demostración, y ningún trabajo del panel pasa por delante de un P0 pendiente. Hojas de vida y seguimiento siguen siendo P2 en el contrato. Y no entra nada nuevo al panel sin una nota que lo apruebe.

Consecuencias

La nota 008 conserva su alcance escrito y remite a esta para la excepción. En la demostración el administrador puede aparecer en pantalla, pero el ensayo (I4) debe ejercer el panel de punta a punta, porque nadie lo ha probado hasta ahora. Si falla, el recorrido de la nota 008 sigue disponible: semilla y llamadas a la API.

La deuda es la que se ve en el código. `AdminDashboard.tsx` y `AdministrativoDashboard.tsx` repiten las mismas llamadas de programación, centros y hojas de vida, y cada una valida los formularios por su cuenta. `cliente-api.ts` acepta un `centroTrabajoId` en `obtenerSeguimientoDia` que el contrato no define (`SeguimientoFiltro` solo tiene `fecha`), aunque ninguna pantalla lo usa. Los defectos que el QA encontró en el cliente del operario no tocan el panel, pero los errores del panel cuentan como defectos del producto. Nada de esto se corrige ahora, porque el plan de correcciones congela las funcionalidades.

014. Autorización por rol en el router, desvío de AGENTS.md

Responsable: A. Decisor: Brayan. Estado: tomada, con deuda.

Problema encontrado

AGENTS.md dice que los roles son administrador, administrativo, supervisor y operario, y que la autorización se decide en la capa de aplicación y no en el router. El código la decide en los routers. Cada ruta de los seis módulos con HTTP declara sus roles con `exigirRol(...)` , un middleware único en `backend/src/infrastructure/http/auth.middleware.ts` que responde 403 `sin_permiso` antes de llegar al caso de uso. Ningún caso de uso recibe ni comprueba el rol de quien llama. El plan de correcciones lo registra como hallazgo H-06 (sección 5) y lo deja como desvío aceptado (sección 8). Hay que fijar la decisión y su razón.

Distinguir dos cosas ayuda a acotar el desvío. La regla sobre el rol dice quién puede llamar a una ruta. La regla sobre el recurso dice sobre qué datos puede actuar quien llama. La segunda vive en la capa de aplicación. En `evidencias` , `subir-evidencia.use-case.ts` rechaza con `SinPermisoError` a un operario que no tiene asignada la actividad, y `obtener-foto-evidencia.use-case.ts` rechaza con `EvidenciaAjenaError` (también `SinPermisoError`) a un operario que pide la foto de otro (hallazgo 1 de la revisión profunda, PR #24). En `jornada` , `registrar-jornada.use-case.ts` rechaza con `ActividadAjenaError` una actividad que no es del operario (tarea B-2).

Opciones evaluadas

1. Mover la comprobación del rol a cada caso de uso, que recibiría el actor como parámetro. Es lo que dice AGENTS.md. Obliga a tocar los casos de uso de los seis módulos y sus pruebas.
2. Mantener la comprobación en el router y registrar el desvío.
3. Comprobar en las dos capas. Descartada: duplica la regla y obliga a mantenerla coherente en dos sitios.

Opción seleccionada y por qué

Opción 2. La regla por rol es una tabla por ruta, declarativa y visible en una línea junto a la ruta, y se contrasta con la columna de acceso de `docs/contrato-api.md` . El único punto de ejecución es el middleware compartido. La prueba de ejecución del QA recorrió las 20 rutas no públicas del registro: todas exigieron token y cada ruta restringida respondió 403 a los roles no permitidos. Desde el 7 de noviembre de 2026 la matriz completa tiene prueba: `backend/src/matriz-roles.test.ts` recorre el registro `rutas` de `shared` y comprueba el 401 sin token y el 403 `sin_permiso` de cada rol no permitido. Antes solo había algunos casos de 403 en `contrato-http.test.ts` e `integration.test.ts` . Mover la regla a los casos de uso es un cambio amplio con un beneficio que hoy no se necesita, mientras el plan de correcciones congela todo lo que no esté en él, y el criterio de AGENTS.md es el cambio mínimo.

El desvío no cubre la propiedad del recurso. Esa comprobación está en la capa de aplicación: en `jornada` (tarea B-2) y en las fotos de `evidencias` , tanto al subirlas como al descargarlas.

Consecuencias

Los casos de uso confían en que el router ya filtró por rol. Si aparece otro punto de entrada, como un proceso programado o un script, hay que repetir la comprobación o mover la regla a la capa de aplicación; esa es la deuda. Las pruebas unitarias de los casos de uso no pueden verificar permisos por rol, porque no los conocen, y la cobertura de roles depende de las pruebas HTTP.

El desvío se acepta como definitivo para v1, y por eso AGENTS.md se corrigió en el mismo cambio que esta nota: ahora dice que el rol se comprueba en el router y que la propiedad del recurso se decide en la capa de aplicación. Si aparece un segundo punto de entrada, esta nota y esa frase son lo primero que hay que revisar.

015. El contrato no fija el dominio del correo de los operarios

Responsable: A. Decisor: Brayan. Estado: tomada; las tareas B-8 y C-9 del plan de correcciones están cerradas.

Problema encontrado

El esquema `UsuarioCrear` de `shared/` exigía que un usuario con rol `operario` tuviera un correo terminado en `@plomada.local`, y la misma regla se repite en el backend (`backend/src/modules/identidad/application/crear-usuario.use-case.ts`) y en el formulario `AdminDashboard.tsx`. La regla entró en los tres sitios con el mismo commit, 6ef0868, cuyo mensaje la presenta como "validación estricta de correo institucional (@plomada.local)". El caso de uso de creación de usuarios existía desde 02dad83 sin ella. Ninguna nota la decidió.

El enunciado pide que a los operarios les llegue por correo la programación semanal y que cada operario tenga un correo institucional. Un correo institucional es uno que la empresa asigna en su propio dominio. `.local` es un sufijo reservado para resolución en redes locales (RFC 6762) y no existe en internet, así que un correo `@plomada.local` no recibe nada fuera de la máquina de desarrollo. La regla sirve para el demo con Mailpit, donde cualquier dirección se acepta, y bloquea el envío real: obliga a que ningún operario tenga su dirección verdadera en el sistema. Además, `docs/arquitectura.md` y las notas no nombran ningún dominio de la empresa, y el contrato no es el lugar donde se conoce.

Opciones evaluadas

1. Mantener `@plomada.local` en el contrato. Descartada: impide el envío real a los operarios, que es un requisito del enunciado.
2. Reemplazarlo por otro dominio fijo. Descartada: el dominio real de la empresa no se conoce y escribirlo en `shared/` lo vuelve a fijar en código.
3. Validar solo que el correo esté bien formado, sin restricción de dominio en el código.
4. Validar el formato y dejar la restricción de dominio como variable de entorno opcional del servidor. Descartada por ahora: nadie ha pedido restringir el dominio, y AGENTS.md pide KISS y YAGNI, es decir, no construir configuración sin un caso real. Si la empresa lo pide, se agrega entonces.

Opción seleccionada y por qué

Opción 3. `UsuarioCrear` en `shared/src/usuarios.ts` valida que el correo esté bien formado, igual para todos los roles, y nada más. Es la regla que cumple el enunciado sin inventar un dominio. Si algún día hace falta restringir dominios, es una política de cada despliegue y pertenece al servidor, no al esquema que comparten el cliente y el servidor. Las pruebas de `shared/src/contrato.test.ts` ya no esperan el rechazo de un correo externo, y comprueban que cualquier correo bien formado sirve en los cuatro roles. `docs/contrato-api.md` queda actualizado.

Consecuencias

Las tareas B-8 y C-9 se cerraron (PR #18 y PR #15). El caso de uso `CrearUsuarioUseCase` y el formulario `AdminDashboard.tsx` ya no comparan el correo con `@plomada.local`, y el contrato, el servidor y el cliente aceptan cualquier correo bien formado. La regla sigue en `shared/src/usuarios.ts` como validación de formato y nada más. No se agregó ninguna variable de entorno.

La semilla y los correos de demostración conservan `@plomada.local`, porque son datos de desarrollo y Mailpit los recibe. La deuda que queda es que nadie ha definido con qué dominio entrarán los operarios reales: lo define la empresa, no el equipo, y mientras tanto el sistema acepta cualquiera.

016. El día de negocio se cuenta en hora de Bogotá

Responsable: A. Decisor: Brayan. Estado: tomada, hallazgo 5 de la revisión profunda.

Problema encontrado

El backend calculaba «hoy», la semana en curso y el día al que pertenece un evento de jornada con la fecha UTC. El despliegue y los usuarios están en Colombia (UTC-5, sin horario de verano), así que a partir de las 19:00 locales la fecha UTC ya es la del día siguiente. Cuatro sitios lo hacían: `obtener-seguimiento-dia.use-case.ts`, `obtener-mi-programacion.use-case.ts`, `pg-registro-jornada.repository.ts` y su versión en memoria. Por la noche el seguimiento sin `fecha` mostraba un día vacío, una salida registrada a las 19:30 aparecía en el día siguiente y, desde el domingo a las 19:00, `/mi-programacion` pasaba a la semana próxima antes de tiempo.

La nota 005 fija que los instantes se guardan en UTC, y eso no cambia. Lo que faltaba era decidir a qué día de calendario pertenece un instante.

Opciones evaluadas

1. Dejar UTC y documentarlo. Descartada: el supervisor ve el día equivocado cada tarde.
2. Zona configurable por variable de entorno. Descartada: nadie la ha pedido y es configuración sin un segundo caso real (YAGNI).
3. Una zona fija de negocio, `America/Bogota`, con un único helper.

Opción seleccionada y por qué

Opción 3. `backend/src/compartido/tiempo.ts` define `ZONA_NEGOCIO` y las funciones `hoyEnNegocio`, `lunesDeLaSemanaActual` y `diaDeNegocio`; los cuatro sitios y el seed las usan, de modo que la regla vive en un solo lugar. En PostgreSQL el filtro por día usa `AT TIME ZONE` con la misma constante. Los instantes siguen en UTC. El mismo archivo trae `esMismoInstante`, que compara dos instantes como instantes y no como texto (hallazgo 7): el contrato acepta `...:00Z` y PostgreSQL devuelve `...:00.000Z`.

Consecuencias

La carpeta `compartido/` está dentro de `backend/src`, fuera de los módulos, y no importa nada salvo `@plomada/shared`; así los casos de uso la usan sin depender de infraestructura. Si el sistema se despliega en otro país, hay que cambiar la constante y revisar esta nota. El cliente tiene su propio helper, `frontend/src/utils/tiempo.ts`, con la misma zona, y desde el 7 de noviembre de 2026 lo usa para decidir qué día muestra la pantalla del operario. Las demás pantallas del cliente todavía calculan "hoy" con la fecha UTC (Anexo C del documento de arquitectura).

Anexo A. Método de priorización

Los atributos y su prioridad se obtuvieron con una versión reducida del taller de atributos de calidad (QAW) que usa el curso. No hubo una sesión con la empresa: el equipo aplicó los pasos usando el enunciado como voz de los stakeholders, y por eso las cifras del ítem 1 se presentan como supuestos.

1. **Presentación.** Se fijó el objetivo: descubrir los atributos de calidad que orientan la arquitectura antes de decidirla.
2. **Objetivos de negocio.** Son las metas del negocio del capítulo de contexto.
3. **Plan arquitectónico.** Se partió de una vista inicial con cliente, servidor, base de datos y correo, que es la vista general de la sección 5.
4. **Conductores arquitectónicos.** La operación sin conexión, la trazabilidad de la evidencia, la seguridad del acceso y los celulares de gama baja.
5. **Generación de escenarios.** Se leyó cada necesidad del enunciado desde cada stakeholder, lo que dio los escenarios funcionales del contexto.
6. **Consolidación.** Los escenarios se agruparon por característica y subcaracterística de ISO/IEC 9126.
7. **Priorización.** Cada atributo recibió una prioridad según cuánto daño hace incumplirlo y cuántas decisiones condiciona, que es la columna de prioridad de las propiedades sistémicas.
8. **Refinamiento.** Los atributos de prioridad alta se convirtieron en los escenarios EC-01 a EC-10, con una medida verificable, y se resumieron en el árbol de utilidades.

Anexo B. Glosario

Vocabulario común del proyecto. Es el único lugar donde se define qué significa cada término; si una palabra del código, de la API o del documento difiere de esta lista, se corrige el código o se propone el cambio aquí con un PR. La columna del identificador indica el nombre exacto en TypeScript o en la API, que es el que se usa en español para el dominio y en inglés solo para los sufijos técnicos.

Personas y lugares

Término	Definición	Identificador
Conjunto cerrado	Cliente de la empresa: propiedad horizontal que contrata el mantenimiento. Cada uno es un centro de trabajo.	
Centro de trabajo	Lugar físico donde se realiza el trabajo, que es el mismo conjunto cerrado.	`CentroTrabajo`
Usuario	Persona con acceso al sistema, identificada por su correo institucional. Tiene un rol.	`Usuario`
Rol	Lo que un usuario puede hacer. Hay cuatro: administrador, administrativo, supervisor y operario.	`Rol`
Administrador	Rol único de la empresa. Gestiona usuarios y programa la semana.	`administrador`
Administrativo	Rol de apoyo (unos 20). Programa y consulta, sin gestionar usuarios.	`administrativo`
Supervisor	Rol de unos 30 usuarios. Revisa el trabajo en sitio y registra su decisión.	`supervisor`
Operario	Rol de unos 150 usuarios. Ejecuta las actividades y sube la evidencia desde su celular personal.	`operario`

Trabajo programado

Término	Definición	Identificador
Programación semanal	Plan de una semana, de lunes a viernes, con todas las asignaciones. Hay una sola por semana y nace en borrador.	`ProgramacionSemanal`
Borrador y publicada	Estados de una programación. Al publicarla se envía el correo a los operarios y deja de editarse.	`EstadoProgramacion`
Asignación	Trabajo de un operario en un centro de trabajo durante un día, con una o más actividades.	`Asignacion`
Actividad	Tarea concreta dentro de una asignación, como limpiar la piscina o pintar un muro. Es la unidad que se evidencia y se revisa.	`Actividad`
Estado de actividad	Pendiente, en curso, finalizada, aprobada o rechazada. Lo deriva el servidor, nunca el cliente.	`EstadoActividad`

Evidencia y revisión

Término	Definición	Identificador
Evidencia	Foto tomada por el operario como prueba del trabajo, ligada a una actividad. Nunca se modifica.	`Evidencia`
Tipo de evidencia	Momento que documenta la foto: antes de empezar, durante el trabajo o después de terminar.	`TipoEvidencia`: `antes`, `durante`, `despues`
Revisión	Decisión del supervisor sobre una actividad: aprobada o rechazada, con observación obligatoria al rechazar.	`Revision`
Cola de revisión	Actividades con al menos una evidencia y sin revisión. Es la lista de trabajo del supervisor.	`revision=pendiente`

Jornada

Término	Definición	Identificador
Registro de jornada	Hecho con hora que el operario marca: entrada o salida del centro, inicio o fin de una actividad.	`RegistroJornada`
Tipo de registro	Cuál de los cuatro hechos es.	`TipoRegistroJornada`: `entrada_centro`, `salida_centro`, `inicio_actividad`, `fin_actividad`
Hora capturada	Momento real del evento según el reloj del celular.	`capturadoEn`, `capturadaEn`
Hora recibida	Momento en que el servidor lo recibió. Puede ser horas después de la capturada.	`recibidoEn`, `recibidaEn`
Hoja de vida	Datos laborales y de formación de un operario.	`HojaDeVida`
Seguimiento	Vista de solo lectura del día: dónde está cada operario y cómo van sus actividades.	`SeguimientoDia`

Sincronización

Término	Definición	Identificador
Modo sin conexión	El cliente funciona sin internet: lee la programación guardada y registra todo localmente.	
Cola de sincronización	Lista local, en el celular, de registros y fotos pendientes de enviar. Sobrevive al cierre de la aplicación.	
Sincronizar	Drenar la cola contra el servidor cuando hay conexión, de a un elemento por vez.	
Idempotencia	Propiedad de una escritura que, repetida con el mismo contenido, deja el sistema igual que si se hubiera hecho una sola vez. Es lo que evita fotos duplicadas tras un reintento.	`PUT` con UUID del cliente

Arquitectura

Término	Definición	Identificador
Módulo	Parte del backend con su propio dominio, casos de uso y adaptadores, que solo se comunica con otros por su interfaz pública.	`backend/src/modules/<modulo>/`
Dominio	Reglas del negocio, sin dependencias externas.	carpeta `domain/`
Caso de uso	Una operación del sistema, como publicar una programación. Un archivo por caso de uso.	carpeta `application/`
Puerto	Interfaz que un caso de uso necesita del exterior, como guardar una foto o enviar un correo.	`application/ports/`
Adaptador	Implementación concreta de un puerto, como PostgreSQL, disco local o SMTP.	carpeta `infrastructure/`
Contrato	Acuerdo entre backend y frontend: esquemas, rutas, roles y reglas. Vive en `shared/` y en `docs/contrato-api.md`.	`@plomada/shared`
Prioridad P0, P1, P2	Qué tan necesaria es una ruta para la demostración: P0 imprescindible, P1 completa el flujo, P2 solo si sobra tiempo.	`Prioridad`

Anexo C. Verificación y defectos conocidos

Este anexo dice qué se comprobó sobre el código final, con qué límites, y qué defectos quedan. Las pruebas, la CI y el guion de aceptación se ejecutaron sobre el commit `943c5ae` de la rama `integracion`, que reúne las correcciones de los PR #24, #25, #26 y #30 y los cambios de documentación y de figuras de los PR #31 y #32.

Pruebas y CI

Paquete	Pruebas	Resultado
`shared`	20	Pasan.
`frontend`	55	Pasan.
`backend`	242	Pasan, las de adaptadores contra una base PostgreSQL real.

`pnpm -r typecheck` no da errores en los cuatro paquetes (`docs`, `shared`, `backend` y `frontend`). La CI del PR #30 (`verificar`: instalación con lockfile congelado, migraciones de prueba, `format:check`, typecheck y pruebas, sobre PostgreSQL 17 y Node 22) pasó antes de fusionarlo. Las pruebas locales corrieron contra PostgreSQL 18 en un clúster temporal. GitGuardian marca cinco hallazgos en el PR #23, que son dos cadenas: la clave con que una prueba firma tokens (`backend/src/adaptadores-pg.test.ts`) y la contraseña de la base desechable de la CI (`.github/workflows/ci.yml`). Las dos son falsos positivos: la primera no se usa fuera de esa prueba y la segunda es la de un contenedor que la CI crea y destruye en cada ejecución.

Guion de aceptación

Se ejecutó el guion de la sección 7 de `docs/plan-de-correcciones.md` con Chromium en emulación de móvil (Pixel 7) sobre la PWA construida (`vite preview`), el backend con `tsx` y PostgreSQL, ya con la corrección de la pantalla del operario (PR #30). No sustituye a una prueba en un celular real.

1. El administrativo creó desde la interfaz una programación para la semana del 9 de noviembre, con una asignación por operario, y la publicó. La semana en curso ya estaba publicada por la semilla. Llegaron 3 correos, uno por operario.
2. Un operario, sin red, registró entrada, inicio de actividad, tres fotos (de 474 a 558 kB tras la compresión), fin de actividad y salida. La barra mostró 7 pendientes y la aplicación no falló.
3. Con red y el servidor detenido durante 80 segundos, la cola conservó los 7 elementos. Hubo 7 intentos fallidos, el primero al abrir la aplicación y los siguientes a los 0, 2, 7, 15, 31 y 64 segundos, y la barra no dijo "Al día".
4. Al levantar el servidor, la cola se drenó sola en 45 segundos y la pantalla del operario pasó de `pendiente` a `finalizada` sin recargar la aplicación. En la base quedaron 3 fotos y 4 registros de jornada, sin identificadores repetidos, y el último registro de jornada se recibió antes que la primera foto. Los reenvíos con el mismo UUID (200 sin duplicar, 409 si cambia el contenido) los cubre `backend/src/contrato-http.test.ts`; no se repitieron a mano.
5. El supervisor vio la actividad en `finalizada` en su cola. Un rechazo sin observación no se aceptó: el cliente mostró "Un rechazo exige una observación obligatoria" y no envió ninguna petición; el 422 del servidor lo comprueban las pruebas de contrato. La aprobación quedó registrada y el seguimiento del 4 de noviembre mostró la entrada y la salida del operario, ambas a las 10:00.
6. Las pruebas y la CI de la sección anterior.

Hubo tres adaptaciones. El receptor de correo fue un servidor SMTP mínimo escrito para la prueba, en lugar de Mailpit. El reloj del navegador del operario se fijó otra vez en el miércoles 4 de noviembre, porque la corrida cayó en sábado, día en que la semilla no agrega actividades para hoy, y la actividad sembrada de ese miércoles era la del

operario que se usó. El panel del administrativo se usó con ventana de escritorio, porque en el ancho de un celular un botón del formulario queda tapado.

Cifras de la prueba de ejecución

El informe de QA del 6 de noviembre de 2026 midió estas cifras, que recoge `docs/plan-de-correcciones.md`:

Medida	Resultado
18 fotos subidas a 1 Mbps	93,75 s (objetivo: menos de 180 s)
p95 en fotos con 30 supervisores consultando	10 ms
Carga inicial de la aplicación	146 kB

Tienen cuatro límites. Se midieron antes de las correcciones de este plan. La carga duró 20 segundos y no los 30 minutos que pide EC-05. Se hicieron en `localhost`, sin latencia de red real, y con la base casi vacía.

Estado de los defectos

Los defectos D1 a D16 son los del informe de QA, con los identificadores que usa el plan de correcciones.

Defecto	Descripción	Estado
D1	La cola descartaba elementos por fallos transitorios	Corregido (C-1).
D2	Secreto JWT por defecto	Corregido (B-1): `leerSecretoJwt` exige 32 caracteres fuera de las pruebas.
D3, D5	Jornada sobre una actividad ajena	Corregido (B-2).
D4	Estado de la actividad calculado de formas distintas	Corregido (B-3): una sola función, `derivarEstado`.
D13	SQL en la raíz de composición, import interno entre módulos y autorización por rol en el router	Parcial. Corregidos el SQL y el import interno (B-3). La autorización por rol sigue en el router, desvío aceptado (nota 014).
D6	Las fotos podían enviarse antes que la jornada	Corregido (C-2).
D7	El operario no veía lo que fallaba	Corregido (C-3).
D8	El cliente daba por buena una respuesta fuera del contrato	Corregido (C-4).
D9	Íconos y favicon de la PWA	Corregido (C-5).
D10	Errores sin la forma del contrato	Corregido (B-4).
D11	Tipo de archivo declarado por el cliente	Corregido (B-7), con el límite que se describe más abajo.
D12	Puerto de las pruebas y CI	Corregido (B-5).
D14	Subida de evidencias no atómica	Corregido (B-7) para la fila; queda el límite del hallazgo 8.
D15	Calidad del repositorio: ESLint, Prettier, cabeceras de seguridad y semilla	Parcial. Corregidos: `X-Powered-By` desactivado (`composition-root.ts`, con prueba en `contrato-http.test.ts`), Prettier aplicado y `pnpm format:check` en la CI. Sin corregir: ESLint y `X-Content-Type-Options`. La semilla sigue vaciando las tablas fuera de producción.
D16	Compresión adaptativa de fotos	No corregido: la compresión usa calidad fija (`frontend/src/utils/compresion-foto.ts:49`).

La revisión profunda del backend (rama `rev/backend`, 7 de noviembre de 2026) dio nueve hallazgos:

Hallazgo	Descripción	Estado
1	Un operario podía descargar la foto de otro	Corregido (PR #24).
2	Dos renovaciones simultáneas con el mismo refresh daban dos sesiones	Corregido (PR #24).
3	El pool de conexiones sin oyente de errores	Corregido (PR #25).
4	La subida multipart sin tope de campos	Corregido (PR #25).
5	El día y la semana se calculaban en UTC	Corregido (PR #26, nota 016).
6	El respaldo de `/mi-programacion` devolvía una semana cualquiera	Corregido (PR #26): la más reciente, y el contrato lo dice.
7	Un reintento idéntico recibía 409 si cambiaba el formato del instante	Corregido (PR #26).
8	Una carrera entre dos `PUT` puede sobrescribir la foto	No corregido (ver más abajo).
9	Violaciones de integridad de la base terminaban en 500	Corregido en parte (PR #25): 23503 responde 404 y 23505 responde 409.

Defectos conocidos sin corregir

- **Autorización por rol en el router.** El rol se comprueba con `exigirRol` en cada router y no en los casos de uso (nota 014). Un punto de entrada nuevo, como un proceso programado, tendría que repetir la comprobación. La matriz completa de roles sí tiene prueba (`backend/src/matriz-roles.test.ts`).
- **Sin eslint ni `X-Content-Type-Options`.** El repositorio no tiene configuración de eslint y el backend no envía esa cabecera. La semilla (`backend/src/seed.ts`) vacía todas las tablas y solo se bloquea con `NODE_ENV=production`.
- **Compresión con calidad fija.** Las fotos se exportan a JPEG con calidad 0,8, sin adaptarla al peso que resulte.
- **La firma de la foto no prueba que la imagen sea válida.** `contenidoCoincideConTipo` (`backend/src/modules/evidencias/domain/firma-archivo.ts`) comprueba los primeros bytes de JPEG y WebP, no que el archivo se decodifique.
- **Sobrescritura de una foto por una carrera.** `SubirEvidenciaUseCase` escribe el archivo en disco antes de insertar la fila. Dos `PUT` del mismo id con bytes distintos pueden hacer que el segundo reemplace el archivo del primero, aunque la base conserve la fila del ganador (hallazgo 8 y límite de D14). Un cliente correcto no envía bytes distintos con el mismo id.
- **Publicaciones simultáneas.** `PublicarProgramacionUseCase` comprueba el estado en memoria y luego actualiza sin condición. Dos publicaciones simultáneas del mismo borrador pueden enviar los correos dos veces (hallazgo 9). Publicar de nuevo, en serie, no reenvía.
- **Identificadores sin validar antes de guardar.** `CrearProgramacion` y `GuardarHojaDeVida` no comprueban que los identificadores existan. Ahora la base responde y el servidor devuelve 404 o 409, no 500.
- **Prueba en celular parcial.** Se hizo por HTTP sobre una IP, así que no demuestra la instalación, el service worker ni el almacenamiento persistente (`docs/pruebas-celular.md`).
- **TLS.** Queda fuera de v1: todo corre por HTTP en `localhost`.
- **Demostración solo de lunes a viernes.** La semilla crea actividades para hoy únicamente en días hábiles, en hora de Bogotá; un sábado o un domingo el operario no tiene nada para hoy.
- **Fechas del cliente en UTC en otras pantallas.** La pantalla del operario elige el día en hora de Bogotá (`frontend/src/utils/tiempo.ts`), pero la semana inicial de esa pantalla y los paneles de administración, administrativo y supervisor siguen calculando "hoy" con la fecha UTC; entre las 19:00 y la medianoche ven el día siguiente.